



كلية العلوم ببئررت
Faculté des Sciences de Bizerte



FASCICULE DE COURS

MACHINE LEARNING2

2^{ème} année Mastère pro Data Science

Semestre 1 ♣



DR. TAOUFIK BEN ABDALLAH



MAÎTRE-ASSISTANT À LA FS-BIZERTE
UNIVERSITÉ DE CARTHAGE



taoufik.benabdallah@fsb.ucar.tn



| OBJECTIFS DU COURS



Objectifs du cours

- ☐ Explorer, manipuler et visualiser les données
- ☐ Construire des modèles prédictifs à partir des données d'apprentissage
- ☐ Maîtriser les principes théoriques et pratiques de certaines techniques de ML (linéaire, non linéaire, à noyau et ensemblistes)
- ☐ Évaluer la performance et l'impact des modèles

Organisation du cours △



💡 CHARGE HORAIRE : 21H COURS+TP

💡 EVALUATION : DS/ EXAMEN

👤 CLASSROOM : Machine Learning2
(CODE: **namcxhv**)



Plan

du cours

Plan du cours **ML pour la prédiction**

- 1** Rappel sur les Fondements du ML
- 2** EAD & Prétraitement de données
- 3** Apprentissage supervisé : Estimation des performances
- 4** Réseaux de neurones : De ML vers DL

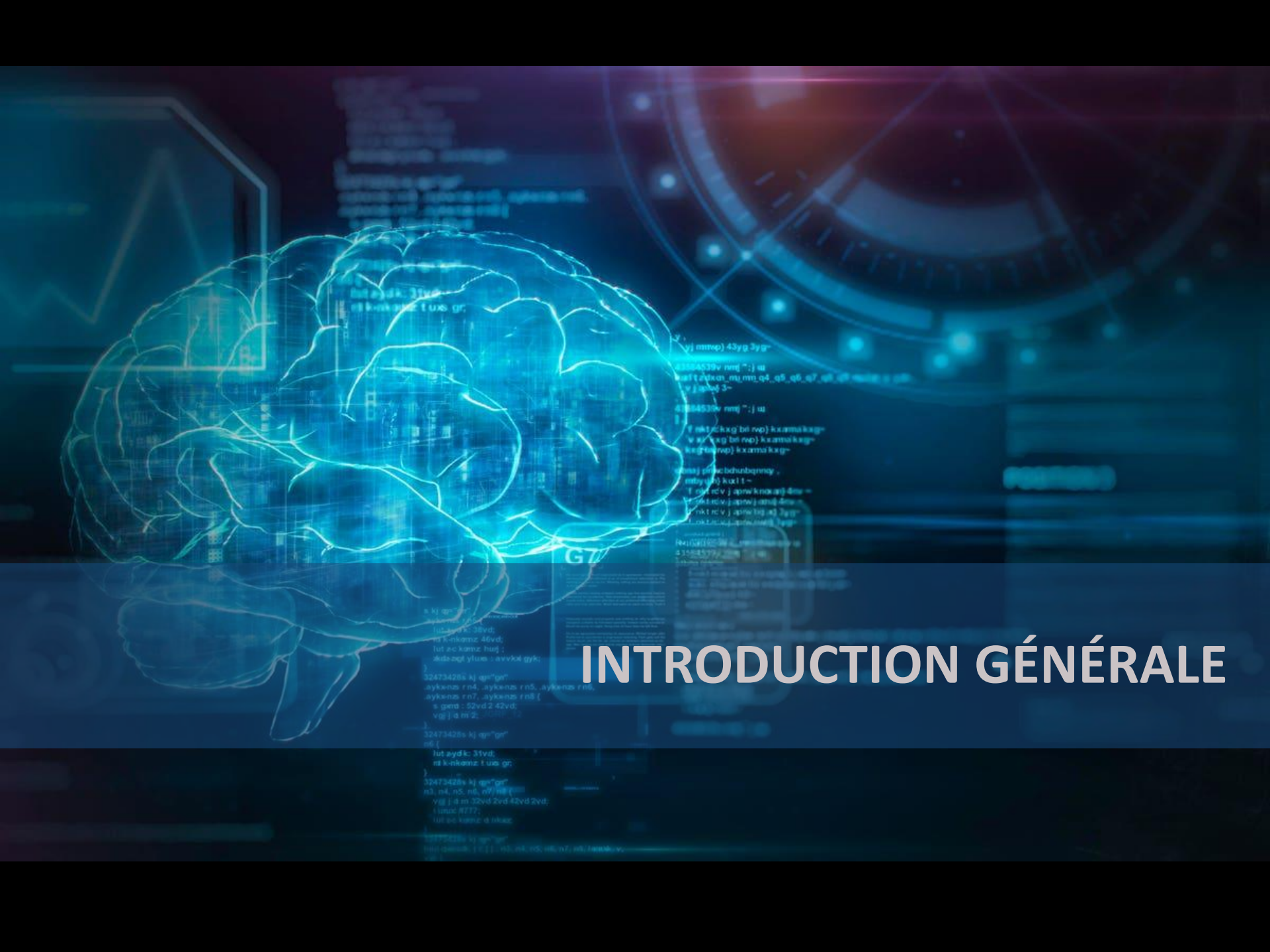




VOLET PRATIQUE

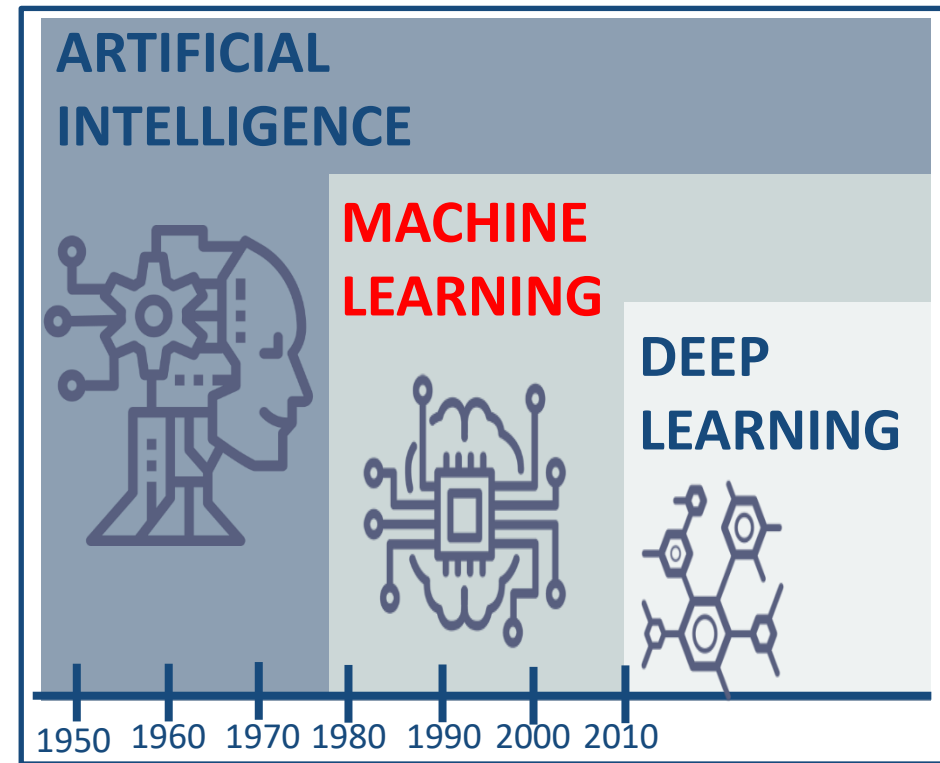
- Calcul Scientifique avec  NumPy
- Exploration des données avec  pandas
- Visualisation graphique avec *matplotlib* & *seaborn*
- Prétraitement de Données, Échantillonnage, Apprentissage Automatique et Évaluation des Performances avec  *scikit-learn*
- Apprentissage des réseaux de neurones avec  TensorFlow





INTRODUCTION GÉNÉRALE

- Artificial Intelligence (AI) : vise à créer des systèmes informatiques capables de simuler des capacités humaines
- Machine Learning (ML) : permet aux ordinateurs d'apprendre à partir de données et de s'améliorer sans programmation explicite
- Deep Learning (DL) : se concentre sur l'entraînement de réseaux de neurones artificiels profonds

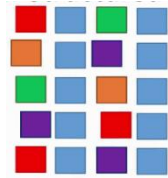


Données?



Données structurées

Données Tabulaire!



Données semi-structurées



{JSON}



Données non-structurées





Fournir les données → Spécifier ce qui doit être accompli → Mesurer la performance!



Expérience

- Prix des maisons
- Images
- Transactions des clients



Tâche

- Prédire le prix
- Grouper les images
- Grouper les clients

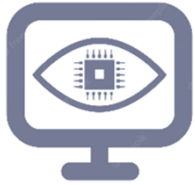


Performance

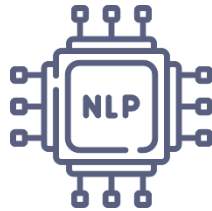
- Prix précis
- Images triées correctement
- Groupement cohérent



Domaines d'application en ML



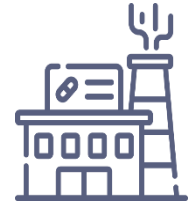
Vision par ordinateur



Traitement du Langage
Naturel (NLP)



Recommandation
de contenu



Industrie



Agriculture



Éducation



Finance



Santé



Transport

Quelques applications en ML



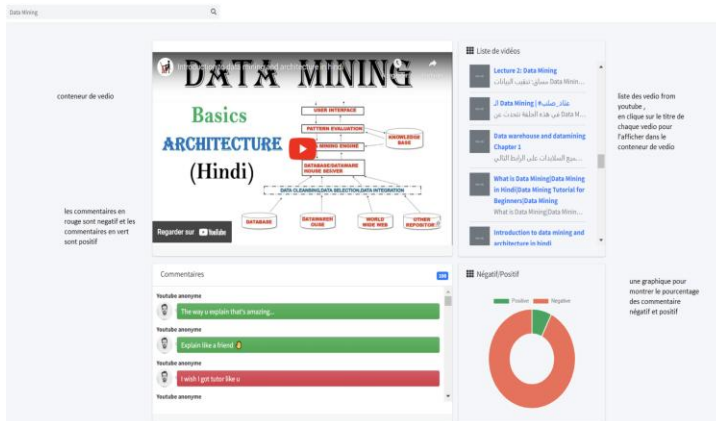
Système de recommandations



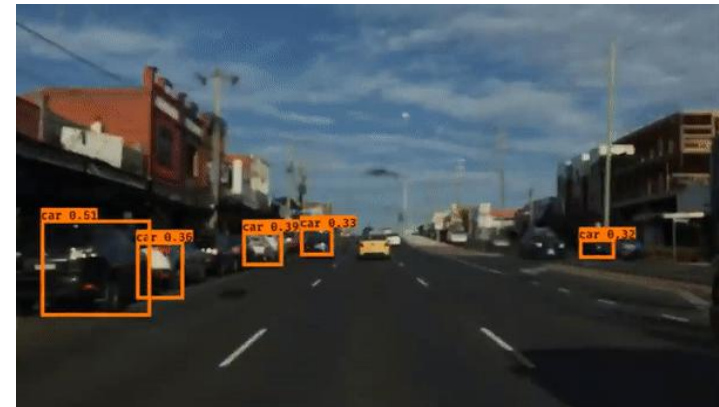
Détection des maladies



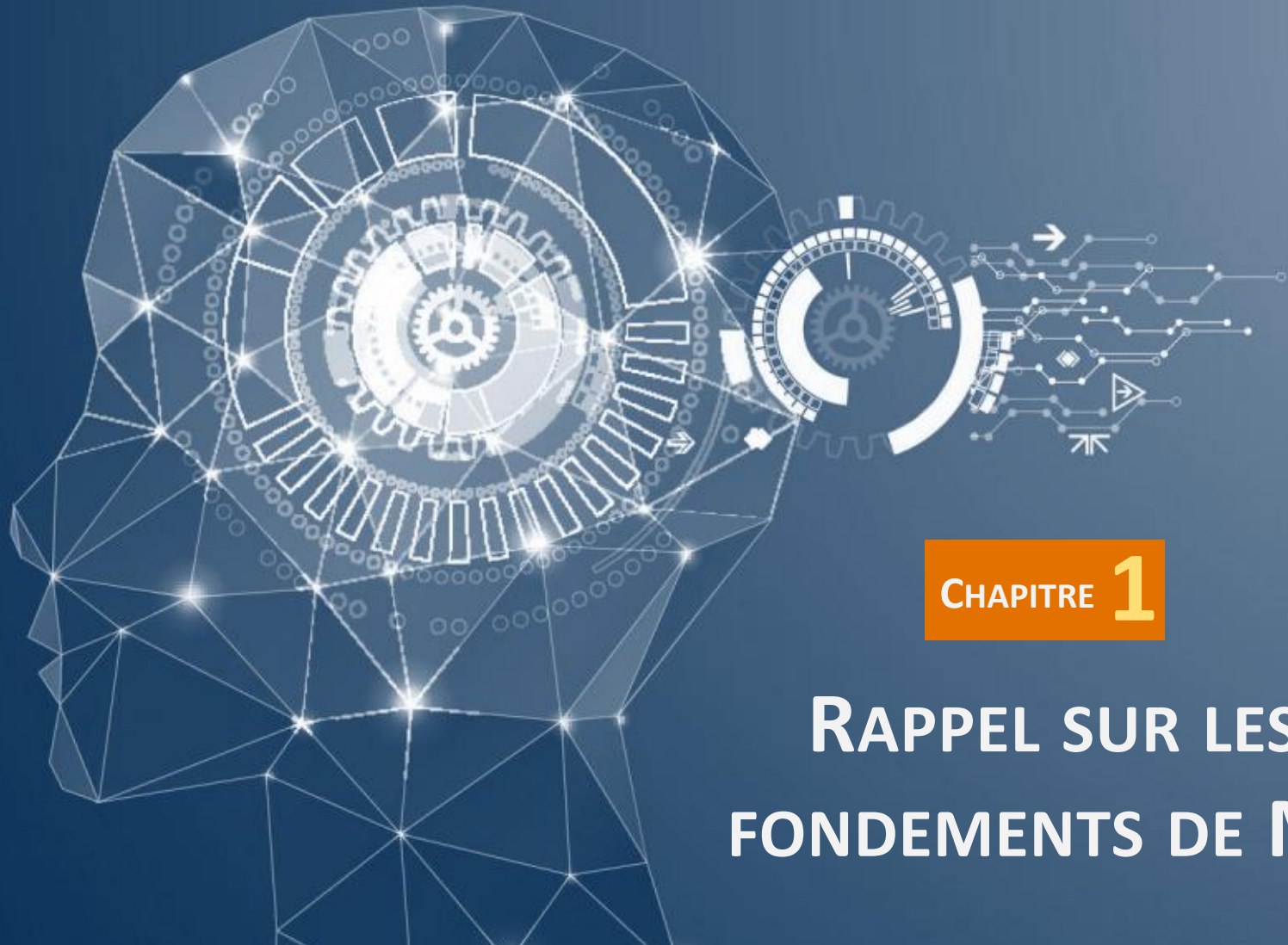
Détection Fraude bancaires



Analyse des avis des internautes



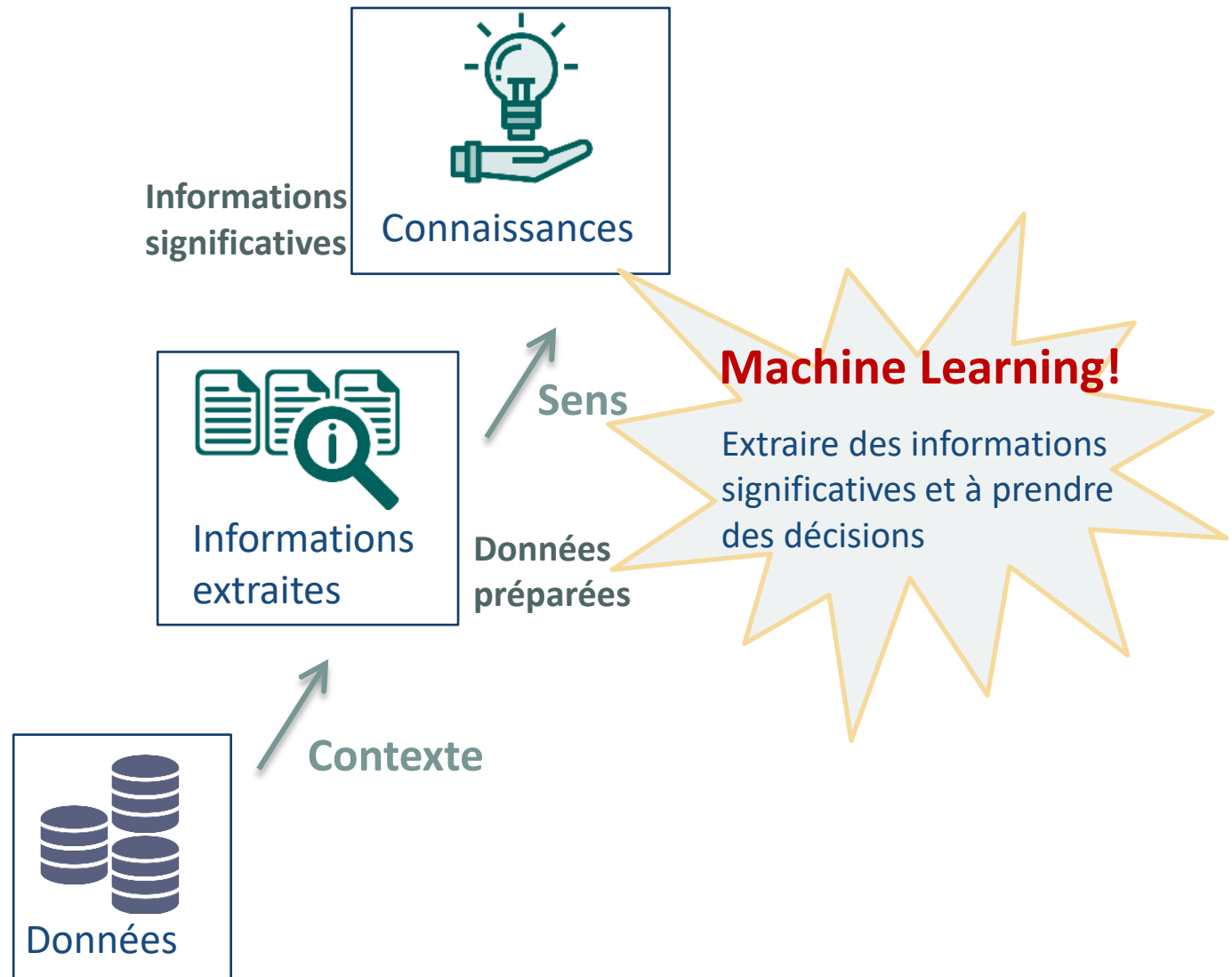
Détection objets en mouvements



CHAPITRE 1

RAPPEL SUR LES FONDEMENTS DE ML

Extraction de Connaissances à partir des Données & ML



| PROCESSUS DE ML



Processus de ML



Phase I: Préparation des données

1. Détection des descripteurs

- ❑ Construction de l'espace des données qui va être exploré
- ❑ Espace des données= Jeu de donnée= **Espace de descripteurs (Feature space)**
- ❑ Un objet est également appelé **échantillon=observation = instance**

Observations	Client	Salaire	S. Familiale	Ville
	→ 1	Moyen	Divorcé	Tunis
	→ 2	Elevé	Célibataire	Tunis
	→ 3	Faible	Célibataire	Sfax
	→ :	:	:	:

← **Descripteurs = Attributs = variables = caractéristiques = Information**

Types de descripteurs

Discret, Binaire ou Continu

Phase I: Préparation des données

Discret : chaîne de caractères

— **Nominal**: Aucune relation existe entre les valeurs

— **Ordinal** : Les valeurs ont un ordre significatif

Types de descripteurs

Continue

Nombres entier ou réels
=valeurs quantitatifs

• **Binaire** : Seulement deux valeurs (vrai, faux / 0,1)

— **Symétrique** : Les deux résultats sont d'importance égale valeurs

— **Asymétrique** : Les résultats n'ont pas la même importance

Exemple de Feature space!

#	Id	Nom	Date. N	Sexe	A1	A2	A3	A4
1	111	John	31/12/1990	M	0	0	Ireland	Dublin
2	222	Mery	15/10/1978	F	1	0.5	Iceland	
3	333	Alice	19/04/2000	F	0	300	Spain	Madrid
4	789	Alex	15/03/2000	A	1	23	Germany	Berlin
5	789	Alex	15/03/2000	A	1	23	Germany	Berlin
6	555	Peter	1983-12-01	M	1	10	Italy	Rome
7	777	Calvin	05/05/1995	M	0	0	Italy	Rome
8	888	Roxane	03/08/1948	F	0	0	Portugal	Lisbon
9	999	Anne	05/09/1992	F	0	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	1	26	Ytali	Rome

Valeur aberrante (Outlier)

Valeur manquante

Lignes dupliquées

Attribut inutile (valeurs uniques)

Format invalide

Valeurs variées

Faute d'orthographe

Phase I: Préparation des données

2. Prétraitement Data preprocessing

Nettoyage de données

- 💡 Remplacer les valeurs manquantes
- 💡 Supprimer les valeurs aberrantes
- 💡 Détecter les anomalies (outliers detection)

Transformation des données

- 💡 Normaliser les données

Discrétisation des données

- 💡 Convertir les attributs continus en attributs discrets

Réduction des données

- 💡 Réduire des données ou des descripteurs

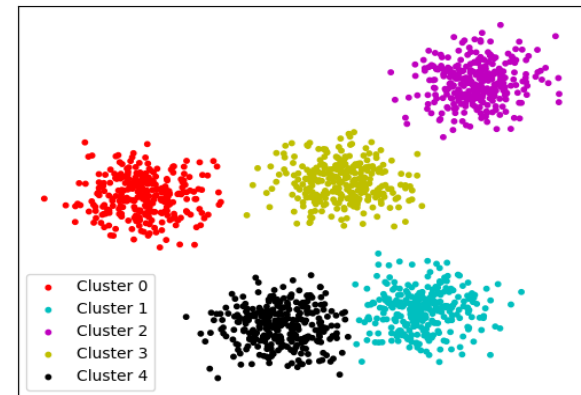
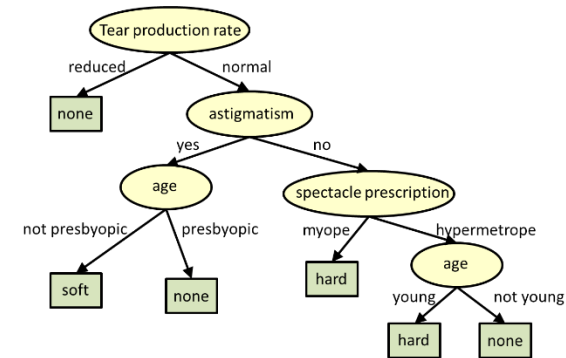


Phase II: Apprentissage automatique

✓ Construction d'un modèle à partir d'un espace de descripteurs

— Le modèle obtenu est une représentation des relations entre les données sous forme de **graphique, fonction, ou règles**

— Un modèle est ensuite utilisé dans le but de **visualisation, description, classification, structuration, explication ou prédiction**



PhaseIII: Validation

✓ **Validation**= évaluer les performances d'un modèle

— Les modèles extraits ne peuvent être utilisés directement en toute fiabilité

📊 Visualisation + interprétation

📋 Métriques d'évaluation (accuracy, loss, etc.)

💡 Améliorer le modèle selon les résultats obtenus

💡 Demander l'avis d'un expert pour valider le modèle

💡 Déploiement & Intégration du modèle



Quiz

1. L'espace de descripteurs est une représentation matricielle (plusieurs réponses)
 - ☐ De connaissances
 - ☒ D'informations
 - ☒ Où les colonnes sont les descripteurs et les lignes sont les échantillons
 - ☐ Où les colonnes sont les échantillons et les lignes sont les descripteurs
2. La préparation de données (plusieurs réponses)
 - ☒ Consiste à déterminer l'espace de descripteurs à utiliser pour l'apprentissage
 - ☒ Nécessite une étape de prétraitement de données
 - ☐ Consiste à donner un sens aux données
 - ☐ Consiste à déployer un modèle d'apprentissage
3. La transformation de données consiste à (seule réponse)
 - ☐ Convertir les attributs continus en attributs sous forme de catégorie
 - ☐ Supprimer les valeurs aberrantes
 - ☒ Normaliser les données pour garantir les mêmes plages de valeurs
 - ☐ Sélectionner les descripteurs les plus discriminantes

| TYPES D'APPRENTISSAGE AUTOMATIQUE



Types d'apprentissage automatique♣



Apprentissage
supervisé



Apprentissage
non-supervisé



Apprentissage
semi-supervisé



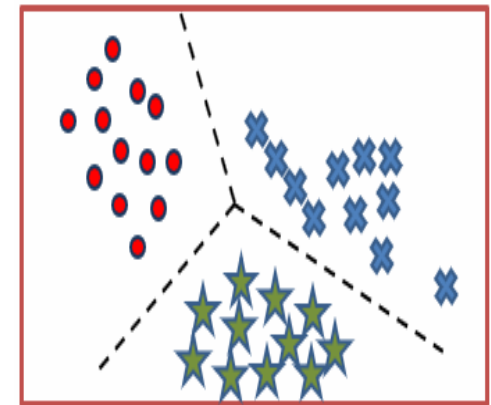
Apprentissage par
renforcement

Apprentissage supervisé (1/4)

✓ Produire automatiquement des règles à partir d'une base de données d'apprentissage étiquetées

— Prédire la classe de nouvelles données observées

Observation	A1	A2	A3	Classe
1	1	0.001	12.134	Chat
2	0	0.023	12.001	Chien
3	1	0.456	9.0073	Chat
4	0	0.114	4.0017	Chat
5	0	0.555	19.771	Chien
6	1	0.551	6.9912	Chat
7	0	0.020	14.478	?
8	1	0.003	8.1238	?



Apprentissage supervisé (2/4)

● Attribut Classe

- Décrit la valeur d'une donnée
- Présente le résultat de la prédiction
- Classe= Attribut spécifique
- *Très couteux à avoir*

✓ **Type de la classe** : peut être de type

Discret, Binaire ou Continu

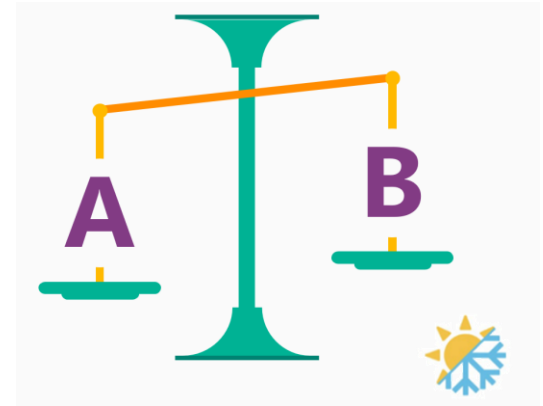
A_1	A_2	...	A_k	...	A_p	Classe
V_{11}	V_{12}					

$k=[1..p]$ avec p est le nombre de descripteurs
= dimension de l'espace de descripteurs

Apprentissage supervisé (3/4)

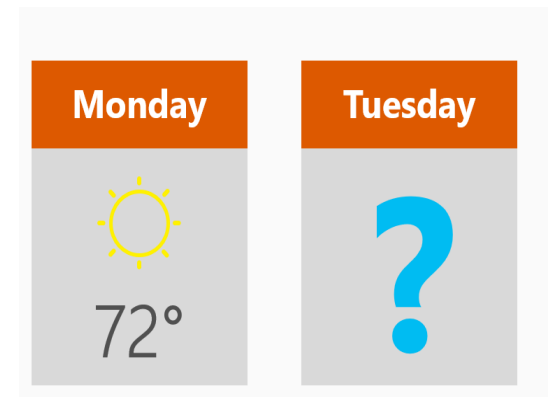
💡 Classification : Prédire des valeurs discrets

- Sera-t-il froid ou chaud demain? **Froid (A)/ chaud (B)**



💡 Régression : Prédire des valeurs continues

- Quelle est la température demain?



Apprentissage supervisé (4/4) Quelques techniques

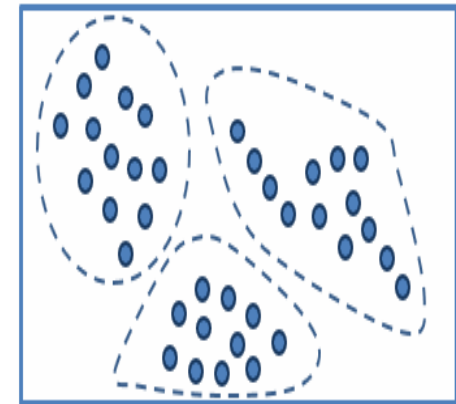
		CLASSIFICATION	RÉGRESSION
	Régression logistique	✓	
	Support Vector Machine (SVM)	✓	✓
	Régression linéaire		✓
	Réseau de neurones	✓	✓
	Arbres de décision	✓	✓
	Random Forest	✓	✓
	Gradient Boosting Decision Tree (GBDT)	✓	✓
	K-Nearest Neighbors (KNN)	✓	
	Naïve Bayes	✓	

Apprentissage non-supervisé (1/3)

✓ Vise à découvrir des structures intrinsèques, des motifs ou des relations inhérentes aux données sans avoir d'informations préalables sur les sorties attendues

— Les classes sont inconnues → pas de cible (pas d'étiquette)

Observation	A1	A2	...	Classe
1	1	12.3	...	Absent
2	0	14	...	
3	0	12	...	
4	0	11	...	
5	1	10.2	...	
6	1	22.1	...	
⋮	⋮	⋮	⋮	



Apprentissage non-supervisé (2/3) **Utilisations**

💡 **Regroupement (Clustering)**

- Organisation des données en groupes (clusters)

💡 **Réduction de dimensionnalité**

- Sélection des descripteurs les plus discriminants
- Transformation des descripteurs dans un autre espace

💡 **Détection d'anomalies (Outliers detection)**

- Identification des observations aberrantes

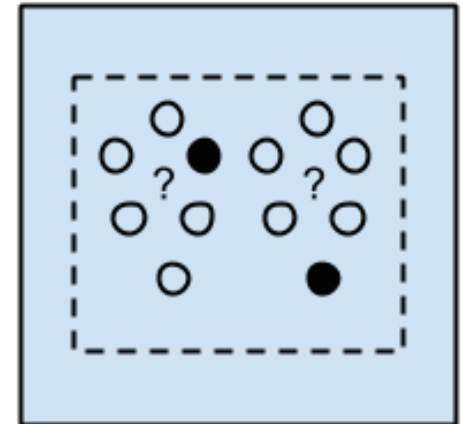


Apprentissage non-supervisé (3/3) Quelques techniques

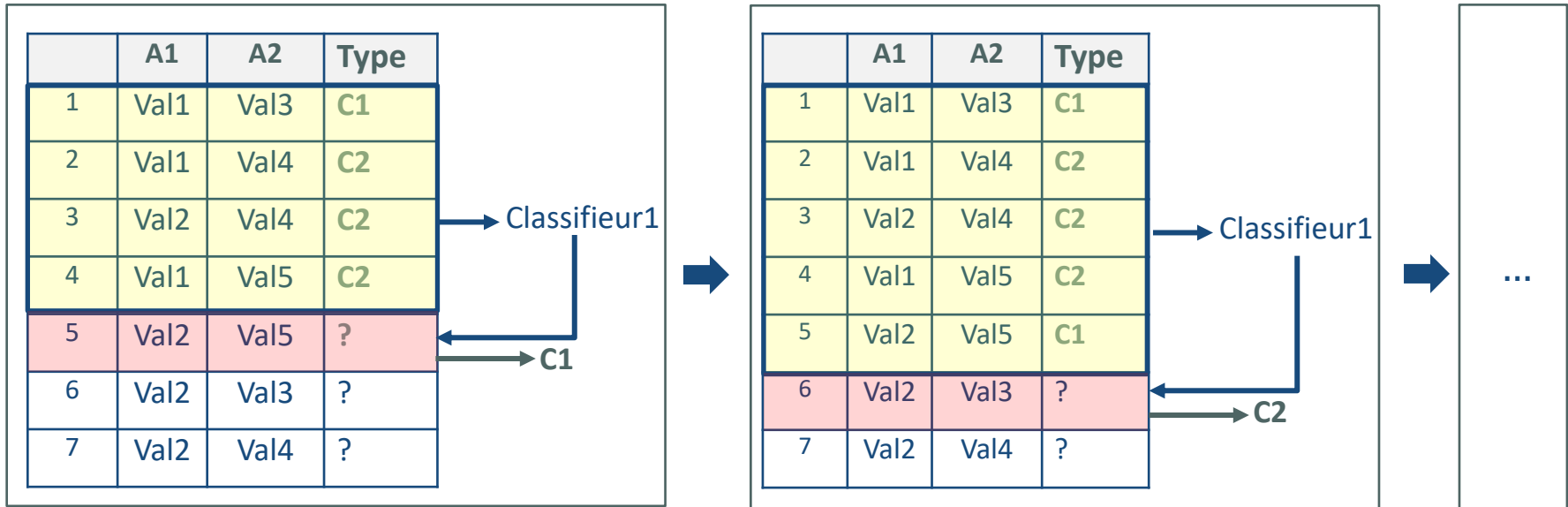
	CLUSTERING	DIMENSIONALITY REDUCTION	OUTLIER DETECTION
 Classification Ascendante Hiérarchique	✓		
 K-Moyenne (K-means)	✓		
 Principal component analysis		✓	
 Isomap		✓	
 One class SVM			✓
 Isolation Forest			✓

Apprentissage semi-supervisé (1/2)

- ✓ Approche hybride qui combine des éléments de l'apprentissage supervisé et non supervisé
 - Permet d'améliorer les performances du modèle en exploitant à la fois les informations des données étiquetées et non étiquetées
 - Inclut souvent des techniques qui exploitent les relations entre les données étiquetées et non étiquetées
 - Peut être particulièrement utile dans des scénarios où les données d'entraînement sont limitées



Apprentissage semi-supervisé (2/2) **Auto-apprentissage**



- a- Entraîner un modèle avec les données étiquetées
- b- Étiqueter une observation incomplète en utilisant le modèle généré dans **a**
- c- Ajouter l'observation étiquetée aux données d'apprentissage
- d- Ré-entraîner le modèle **a** sur les nouvelles données d'apprentissage
- e- Répéter les étapes **a**, **b** et **c** jusqu'à satisfaire un critère d'arrêt (modèle performant)



Apprentissage par renforcement

✓ Apprend à prendre des décisions en explorant l'environnement

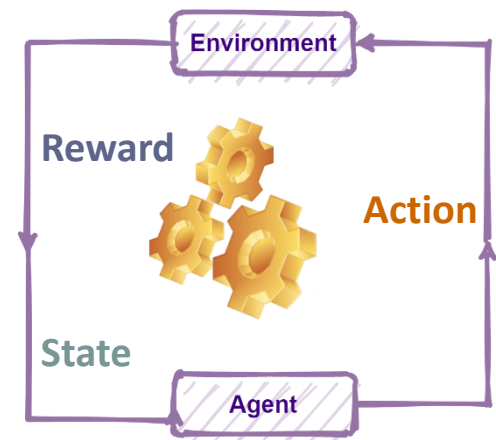
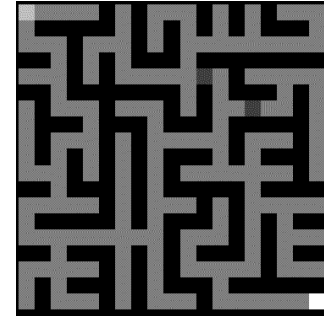
💡 Maximiser une notion de récompense cumulée au fil du temps

💡 Implique un processus d'interaction dynamique entre l'agent et son environnement

— **State**= situation particulière dans laquelle l'agent se trouve

— **Action**= choix que l'agent peut faire à chaque étape (décision prise)

— **Reward**= valeur numérique qui mesure la qualité de la décision prise par l'agent



1. L'apprentissage supervisé (plusieurs réponses)

- ☐ Est appliqué pour constituer des groupes d'objets homogènes et différenciés
- ☐ Consiste à utiliser des données brutes pour prédire des connaissances
- ☒ Est appliqué principalement pour la prédiction
- ☒ Nécessite des échantillons étiquetés par un ou plusieurs classes

2. L'apprentissage semi-supervisé (seule réponse)

- ☐ Minimise l'erreur sur la base de test
- ☐ Consiste à prédire une variable continue
- ☐ Consiste à apprendre les actions, à partir d'expériences
- ☒ Utilise les données étiquetées pour compléter les données non-étiquetées

3. Quelle est la technique qui n'appartient pas à l'apprentissage supervisé ? (seule réponse)

- ☐ Réseaux de neurones
- ☐ Arbre de décision
- ☐ Random Forest
- ☒ K-means

Librairies & Framework





CHAPITRE 2

EDA & PRÉTRAITEMENT DE DONNÉES



Exemple du dataset (avec anomalies!)

		Attribut date			Outliers!!		Attributs discrets	
#	Id	Nom	Date. N	Sexe	A1	A2	A3	A4
1	111	John	31/12/1990	M	<u>767</u>	0	Ireland	Dublin
2	222	Mery	15/10/1978	F	22	0.5	Iceland	NAN
3	333	Alice	19/04/2000	F	60	300	Spain	Madrid
4	789	Alex	15/03/2000	A	35	23	Germany	Berlin
5	789	Alex	15/03/2000	A	15	23	Germany	Berlin
6	555	Peter	1983-12-01	M	NAN	10	Italy	Rome
7	777	Calvin	05/05/1995	M	32	0.13	Italy	Rome
8	888	Roxane	03/08/1948	F	44	0.8	Portugal	Lisbon
9	999	Anne	05/09/1992	F	<u>986</u>	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	24	26	Ytali	Rome

Lignes dupliquées →

valeurs uniques

Valeurs variées

EDA & Prétraitement de données

EXPLORATORY DATA ANALYSIS (EDA)

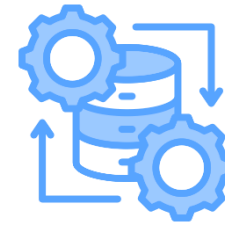
ANALYSE EXPLORATOIRE DES DONNÉES



Analyse les données pour obtenir des informations initiales et comprendre leurs caractéristiques

DATA PREPROCESSING

PRÉTRAITEMENT DES DONNÉES



Prépare et transforme les données afin qu'elles soient prêtes pour l'entraînement des modèles



— EDA & Prétraitement de données sont *complémentaires*

— EDA permet de mieux comprendre les données, ce qui peut orienter les décisions à prendre lors du prétraitement des données

EDA & Prétraitement de données (//)



EXPLORATORY DATA ANALYSIS (EDA)

ANALYSE EXPLORATOIRE DES DONNÉES

- **Exploration initiale** : Identification des variables et les types de données
- **Statistique descriptives**: Aperçu initial de la distribution de données
- **Identification des valeurs manquantes/outliers et duplicata**
- **Visualisation des distributions**
- **Etude de relation entre les attributs** : Analyse de corrélation



DATA PREPROCESSING

PRÉTRAITEMENT DES DONNÉES

- Nettoyage des données
- Transformation de données
- Discrétisation des données
- Réduction des données

Exploration initiale

DataFrame df=

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden
4	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
271111	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
271112	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271113	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271114	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
271115	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

271116 rows × 7 columns

Identification des
types de colonnes

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ID           271116 non-null  int64
1   Name         271116 non-null  object
2   Sex          271116 non-null  object
3   Age          261642 non-null  float64
4   Height       210945 non-null  float64
5   Weight       208241 non-null  float64
6   Team         271116 non-null  object
dtypes: float64(3), int64(1), object(3)
memory usage: 14.5+ MB
```

Age, Height & Weight comportent
des valeurs manquantes!

Statistique descriptives

Attributs de type **objet**

```
df.describe(include="all")
```

	ID	Name	Sex	Age	Height	Weight	Team
count	271116.000000	271116	271116	261642.000000	210945.000000	208241.000000	271116
unique	NaN	134732	2	NaN	NaN	NaN	1184
top	NaN	Robert Tait McKenzie	M	NaN	NaN	NaN	United States
freq	NaN	58	196594	NaN	NaN	NaN	17847
mean	68248.954396	NaN	NaN	25.556898	175.338970	70.702393	NaN
std	39022.286345	NaN	NaN	6.393561	10.518462	14.348020	NaN
min	1.000000	NaN	NaN	10.000000	127.000000	25.000000	NaN
25%	34643.000000	NaN	NaN	21.000000	168.000000	60.000000	NaN
50%	68205.000000	NaN	NaN	24.000000	175.000000	70.000000	NaN
75%	102097.250000	NaN	NaN	28.000000	183.000000	79.000000	NaN
max	135571.000000	NaN	NaN	97.000000	226.000000	214.000000	NaN

Valeur la plus fréquente

Identification des valeurs manquantes NaN

```
df.isnull().sum()
```

Équivalent à `df.isna()`

```
ID          0
Name         0
Sex          0
Age        9474
Height     60171
Weight     62875
Team         0
dtype: int64
```

Nombre total de valeurs manquantes

```
df.isnull().sum().sum()
```

```
#9474+60171+62875=132520
```

df=

Valeurs
manquante

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden
4	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
271111	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
271112	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271113	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271114	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
271115	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

Nettoyage des données : Traitement des NaN



Suppression des instances/ colonnes
ayant des valeurs manquantes

dropna



Remplissage des valeurs
manquantes



Par une valeur
fictive

fillna
bfill
ffill

Par l'élément le plus fréquent/
moyenne/médiane

SimpleImputer

Traitement des NaN **Suppression**



```
df.dropna(subset=["Height", "Weight"],  
          axis=0, inplace=True,  
          ignore_index=True)
```

Valeur par défaut

```
df.dropna(subset="Age", axis=1, inplace=True)
```

**Conserver uniquement les
lignes/colonnes avec au moins
2 valeurs non manquantes!**

```
df.dropna(thresh=2, how="any", inplace=True)
```

```
df.dropna(thresh=2, how="any", inplace=True)
```

df=

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
3	5	Christine Jacoba Aaftink	F	25.0	185.0	82.0	Netherlands
4	5	Christine Jacoba Aaftink	F	27.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
146744	135568	Olga Igorevna Zyuzkova	F	33.0	171.0	69.0	Belarus
146745	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
146746	135570	Piotr ya	M	27.0	176.0	59.0	Poland
146747	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
146748	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

- "any" : Supprimer la ligne ou la colonne si **au moins une valeur** est NaN
- "all" : Supprimer la ligne ou la colonne uniquement si **toutes les valeurs** sont NaN

Traitement des NaN Remplissage (1/2)

✓ Par une valeur fictive

Nombre de NaN total à remplir!

```
df['Age'].fillna(0, inplace=True, limit=2)

df['Age'].fillna(df[['Height', 'Weight']].apply(
    lambda x: 40 if (x[0] >= 170 and x[1] >= 80) else 25,
    axis=1), inplace=True)
```

df=

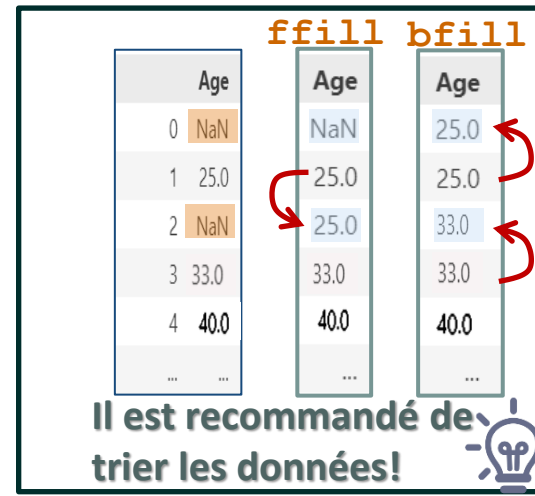
	Age	Height	Weight	Age	Age
0	NaN	180.0	80.0	0.0	40.0
1	NaN	170.0	60.0	0.0	25.0
2	NaN	185.0	82.0	NaN	40.0
3	NaN	185.0	82.0	NaN	40.0
4	NaN	185.0	82.0	NaN	40.0
...	...	...	...	...	...

```
df['Age'].ffill(inplace=True)
df['Age'].fillna(method='ffill', inplace=True)
```

Remplissage
vers l'avant

```
df['Age'].bfill(inplace=True)
df['Age'].fillna(method='bfill', inplace=True)
```

Remplissage
vers l'arrière



Traitement des NaN Remplissage (2/2)

✓ Par l'élément le plus fréquent/ moyenne/médiane

```
from sklearn.impute import SimpleImputer
mf_imputer = SimpleImputer(missing_values=np.nan,
                             strategy='most_frequent')
arr=mf_imputer.fit_transform(df.loc[:, ['Age']])
ndarray

pd.DataFrame(arr, columns=['Age'])
```

↑
Il faut avoir un
DataFrame ou ndarray
2D!

Age	Age
24	24.0
24	24.0
NaN	24.0
25	25.0
30	30.0

'most_frequent'

24.0

strategy='mean' / strategy='median'

Toutes les valeurs des attributs doivent être de type continue

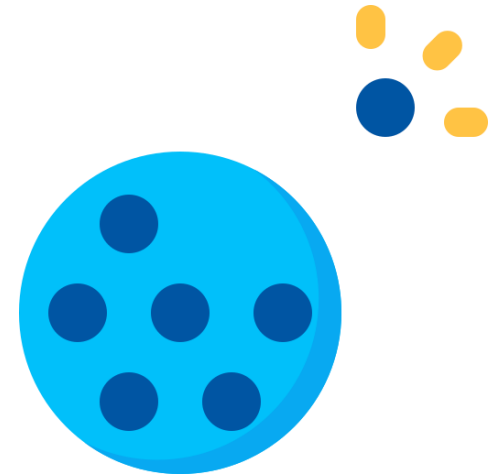


Identification des valeurs aberrantes



Identifier les instances ayant un comportement non conforme

- Méthode des valeurs extrêmes (Extreme Value Analysis)
- Méthode de la règle des k -voisins les plus proches
- Boîtes à moustaches (**Boxplot**)
- Méthodes basées sur des seuils
- Méthode de la fréquence
- Isolation Forest



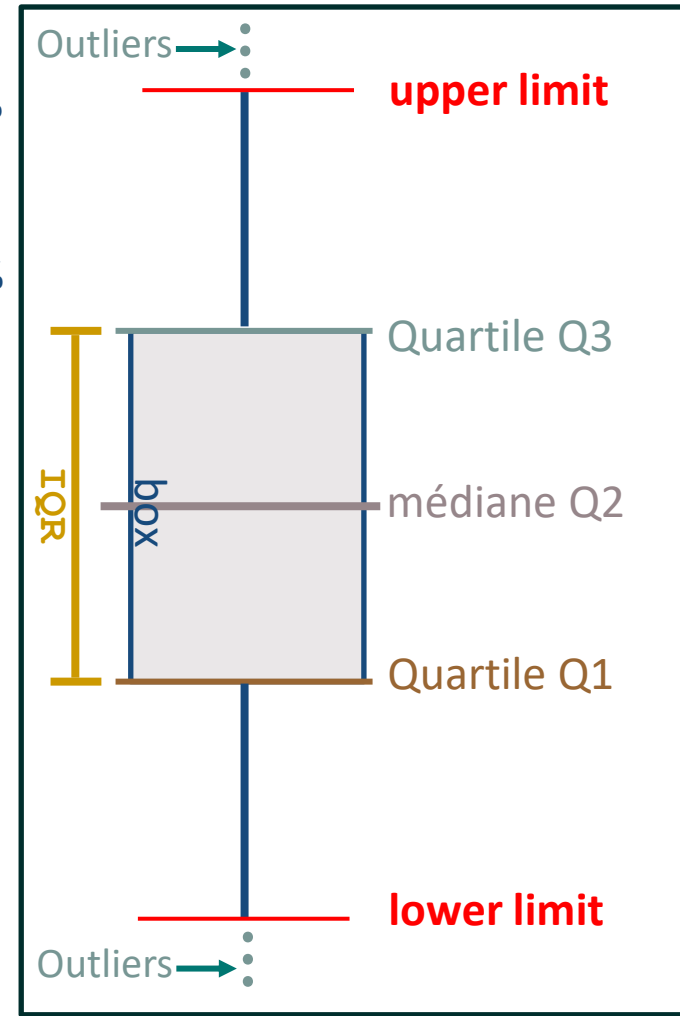
Identification des valeurs aberrantes **Boxplot**

- Premier quartile $Q1 = 25^{\text{e}}$ percentile = 25%
des données sont inférieures à la valeur de $Q1$
- Deuxième quartile $Q3 = 75^{\text{e}}$ percentile = 75%
des données sont inférieures à la valeur de $Q3$
- Médiane = $Q2 = 50^{\text{e}}$ percentile
- Interquartile range (IQR) = $Q3 - Q1$



lower limit = $\max(\min, Q1 - 1.5 \times IQR)$

upper limit = $\min(\max, Q3 + 1.5 \times IQR)$



Boxplot Q1, Q3 ?

Soit la distribution de données **T** suivante : **N=8 observations**

Valeur	10	15	20	25	30	35	40	45
Position	0	1	2	3	4	5	6	7

Q1, Q3= np.percentile(T, [25, 75])

$$\text{Index_Q1} = \left(\frac{25}{100}\right) \times (N-1) = 1.75$$

$$\text{Index_Q3} = \left(\frac{75}{100}\right) \times (N-1) = 5.25$$

$$\text{Q1} = 15 + (20 - 15) \times \text{partie décimale}(\text{Index_Q1}) = 15 + (20 - 15) \times 0.75 = 18.75$$

$$\text{Q3} = 35 + (40 - 35) \times \text{partie décimale}(\text{Index_Q3}) = 35 + (40 - 35) \times 0.25 = 36.25$$

Nettoyage des données : Traitement des outliers

```
sns.boxplot(data=df, x="Height")
```

```
Q1, Q3=np.percentile(df["Height"],  
                      [25, 75])
```

```
IQR=Q3-Q1
```

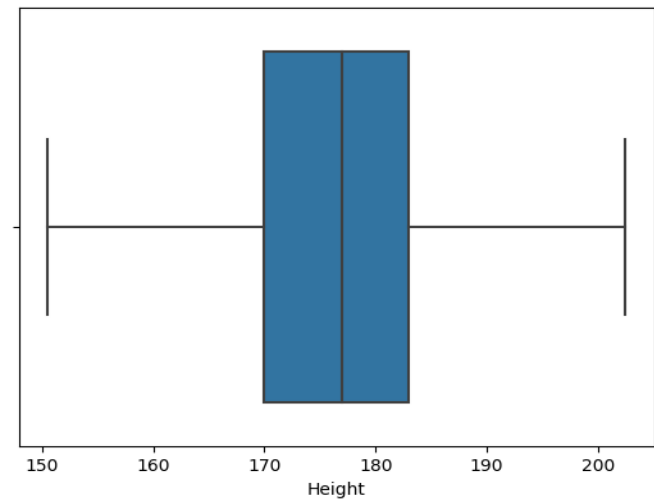
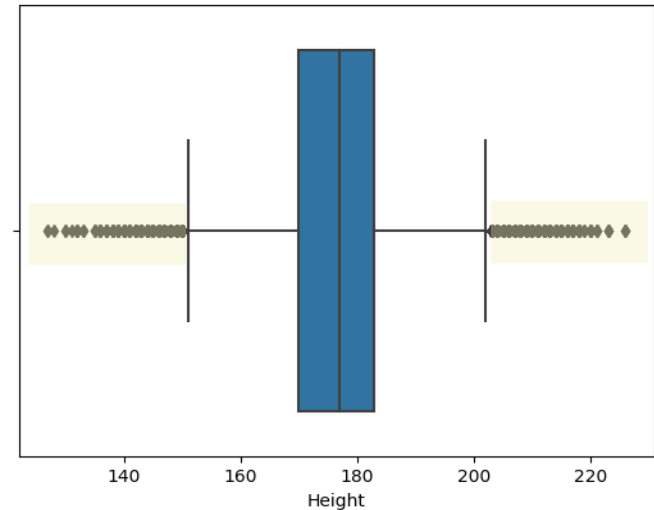
```
upper_limit=Q3+1.5*IQR
```

```
lower_limit=Q1-1.5*IQR
```

```
outliers=df["Height"][(df["Height"]>upper_limit)|  
                       (df["Height"]<lower_limit)]
```

↑
Série

```
df['Height']=np.where(df['Height']>=upper_limit,  
                      upper_limit, np.where(df['Height']<=lower_limit,  
                                              lower_limit, df['Height']))
```



Identification & Traitement des lignes dupliquées

● Détection des lignes dupliquées

```
df1=df[df.duplicated() ]
```

df=

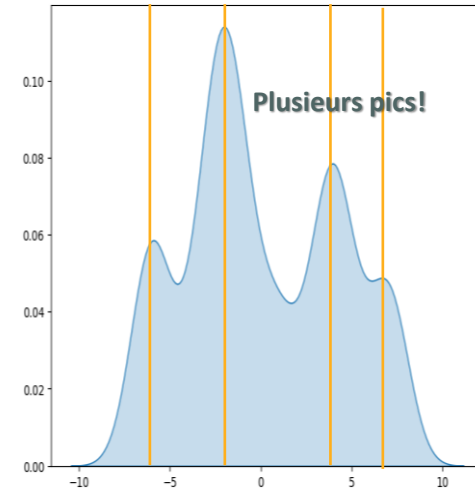
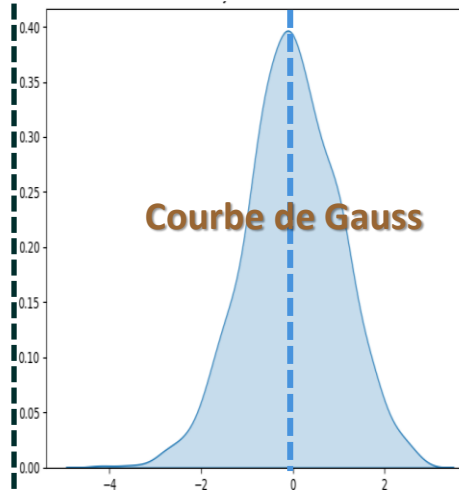
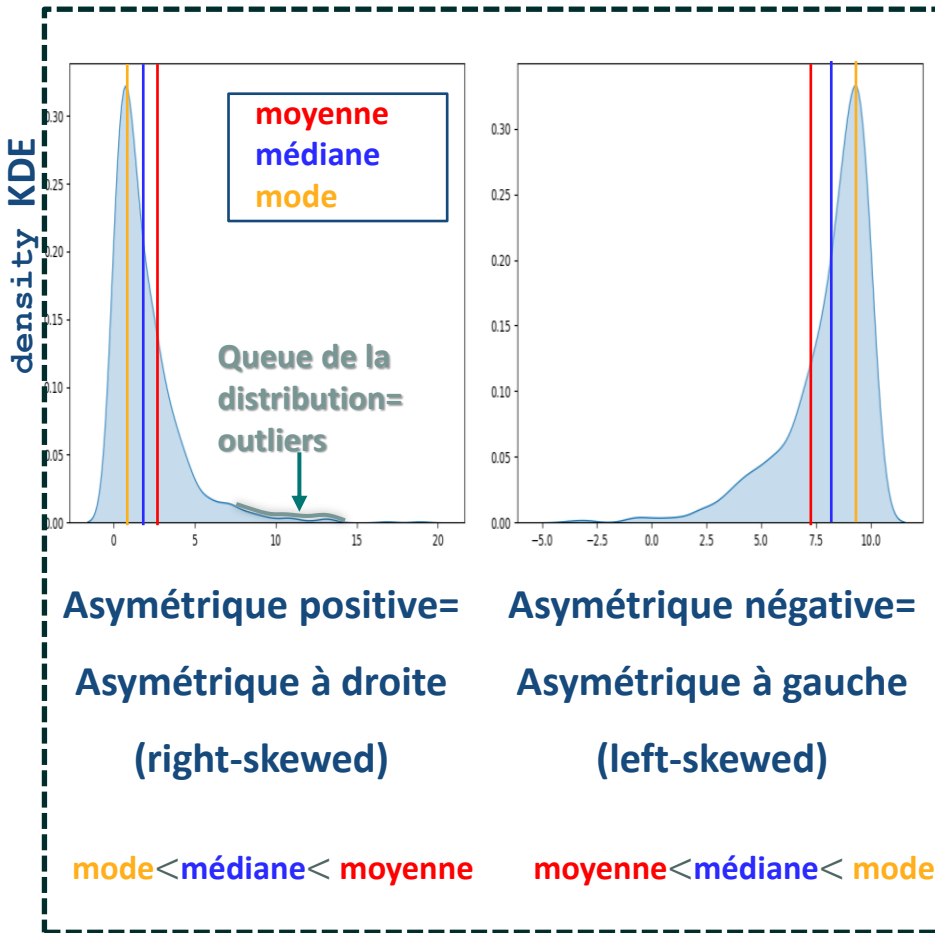
	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	1	A Dijiang	M	24.0	180.0	80.0	China
2	2	A Lamusi	M	23.0	170.0	60.0	China
3	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
4	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden

● Suppression des lignes dupliquées

```
df.drop_duplicates(inplace=True,  
                    ignore_index=True)
```

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden

Type de distribution de données continues



Visualisation des distributions **Attribut continu**



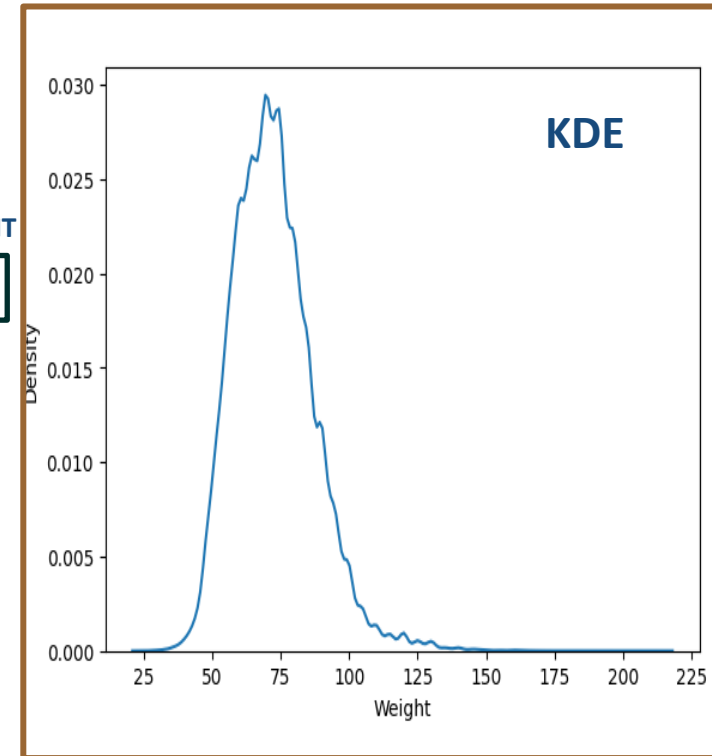
Densité avec une partition gaussienne =Kernel Density Estimator(KDE)

```
sns.kdeplot(df['Weight'],  
            bw_method='scott',  
            bw_adjust=1)
```

FACTEUR D'AJUSTEMENT

💡 **bw_adjust >1**: Estimation de densité plus lisse
(courbe plus lissée avec moins de détails)

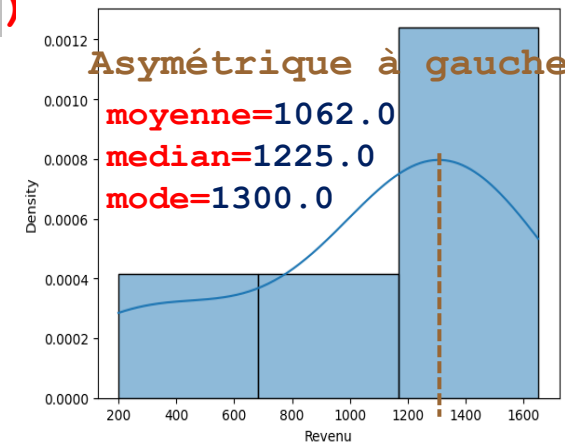
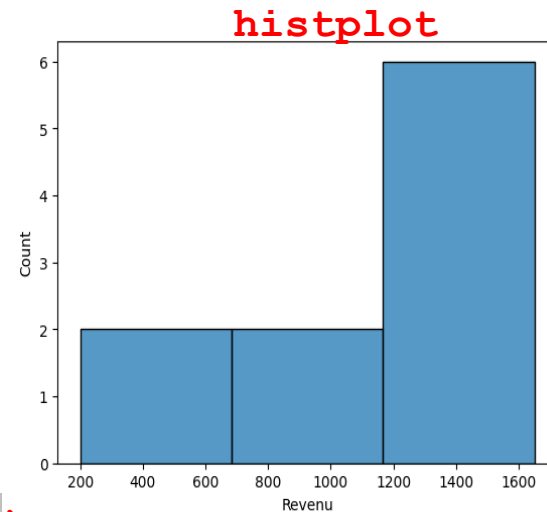
💡 **bw_adjust <1**: Estimation de densité moins lisse
(courbe plus détaillée avec plus de variation)



Histogramme de données continues **histplot+kde**

```
sns.histplot(x="Revenu", data=df,  
             bins=3)
```

```
sns.histplot(x="Revenu", data=df,  
             bins=3, kde=True, stat='density')
```



N=10

df= Revenu

0 $x_1 = 300$

1 $x_2 = 1100$

2 1200

3 1500

4 200

5 1400

6 1250

7 1300

8 1650

9 720

histplot+kde/ choix optimal de nombre de bins!

Règle de Freedman-Diaconis

$$\text{nombre de bins} = 2 \times \text{IQR} \times N^{(-\frac{1}{3})}$$

Règle de Scott

$$\text{nombre de bins} = 3.5 \times \frac{\text{std}(\Omega)}{N^{(\frac{1}{3})}}$$

Règle de racine carrée

$$\text{nombre de bins} = \sqrt{N}$$

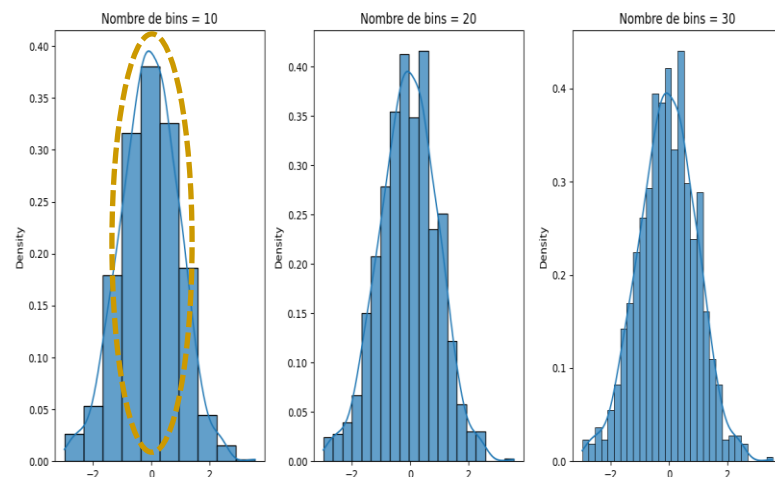
Règle de Sturges (optimale pour des échantillons de taille limitée)

$$\text{nombre de bins} = 1 + \log_2(N)$$

Expérimentation : Essayer plusieurs valeurs de bins & choisir celle qui offre la représentation la plus informative de la distribution des données

Dataset Ω

	X_1	...	X_K
obs1	a_{11}		a_{1K}
$\vdots$	$\vdots$	$\vdots$	$\vdots$
obsi	a_{i1}		a_{iK}
obsN	a_{N1}		a_{NK}

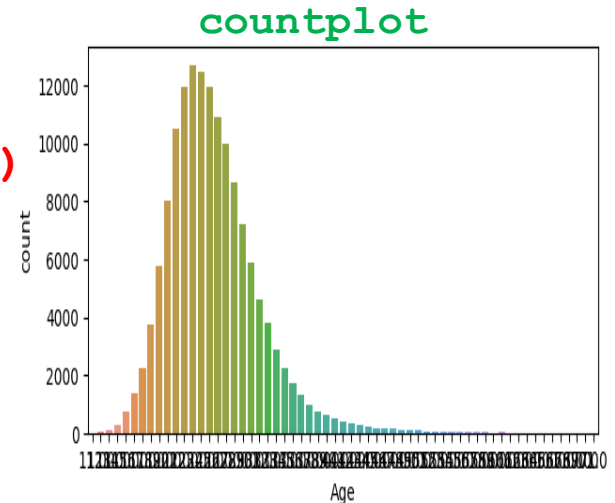


Un choix judicieux du nombre de bins peut donner un histogramme qui reflète bien la distribution!

Histogramme de données discrètes **countplot**

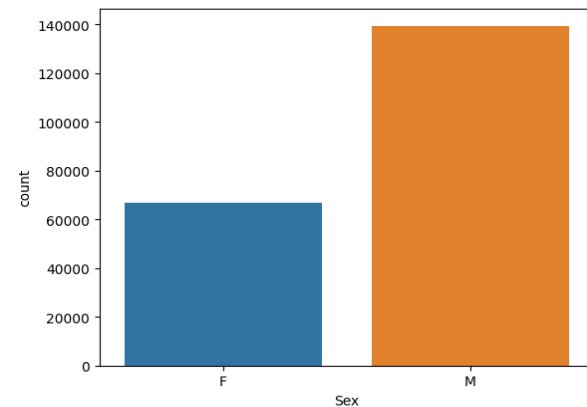
Attribut continu

```
sns.countplot(x="Age", data=df, bins=3)
```



Attribut discret

```
sns.countplot(x="Sex", data=df,  
              order=["F", "M"])
```



df=	Sex	Age
0	M	24.0
1	M	23.0
4	F	21.0
5	F	21.0
6	F	25.0
...	...	...
271111	M	29.0
271112	M	27.0
271113	M	27.0
271114	M	30.0
271115	M	34.0

Transformation des données *numériques* 1/3

Standardization Min – Max

- Transformation des valeurs de caractéristiques à l'échelle pour qu'elles se situent entre un maximum et un minimum

$$a_{ji} = \frac{a_{ji} - \min(\mathbf{x}_i)}{\max(\mathbf{x}_i) - \min(\mathbf{x}_i)} \quad [0 \dots 1]$$

```
from sklearn.preprocessing import MinMaxScaler  
  
min_max_scaler=MinMaxScaler()  
  
arr=min_max_scaler.fit_transform(df)  
  
pd.DataFrame(arr,columns=['X1', 'X2'])
```

	x_1	...	x_i	x_p
obs1	a_{11}	$\vdots$	a_{1i}	a_{1p}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
obsj	a_{j1}	$\vdots$	a_{ji}	a_{jp}
obsN	a_{N1}	...	a_{Ni}	a_{Np}

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après MinMaxScaler

	X1	X2
0	1.000000	0.555556
1	0.000000	1.000000
2	0.571429	0.000000

- 😊 Optimum pour distribution uniforme
- 😊 Préserve la forme de la distribution
- 😞 Sensibles aux valeurs aberrantes

Transformation des données *numériques* 2/3

☰ Standardization zscore z_{ji}

— Transformation des données en divisant la différence entre les données a_{ji} et la moyenne M_i par l'écart type σ_i $z_{ji} = \frac{a_{ji} - M_i}{\sigma_i}$

avec $M_i = \frac{1}{N} \sum_{j=1}^N a_{ji}$ et $\sigma_i = \sqrt{\frac{\sum_{j=1}^N (a_{ji} - M_i)^2}{N}}$ et N= Nombre d'observations

```
from sklearn.preprocessing import StandardScaler
standard_scaler= StandardScaler()
arr= standard_scaler.fit_transform(df)
pd.DataFrame(arr,columns=['X1', 'X2'])
```

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après MinMaxScaler

	X1	X2
0	1.162476	0.090536
1	-1.278724	1.176965
2	0.116248	-1.267500

😊 Optimum pour distribution symétrique

☹ Sensibles aux valeurs aberrantes

Transformation des données *numériques* 3/3

Standardization RobustScaler

— Transformation des données en utilisant la médiane et l'écart interquartile (**IQR**)

$$z_{ji} = \frac{a_{ji} - Q2}{IQR} \text{ avec } IQR = Q3 - Q1 ; Q1 = 25\text{e percentile} ; Q3 = 75\text{e percentile} \\ \text{et } Q2 = 50\text{e percentile} = \text{médiane}$$

```
from sklearn.preprocessing import RobustScaler
standard_scaler= RobustScaler()
arr= standard_scaler.fit_transform(df)
pd.DataFrame(arr,columns=['X1', 'X2'])
```

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après RobustScaler

	X1	X2
0	0.857143	0.000000
1	-1.142857	0.888889
2	0.000000	-1.111111

😊 Optimum lorsqu'il y a des valeurs aberrantes dans les données

Transformation des données *discrètes (Encoding)* 1/3

OrdinalEncoder & LabelEncoder 1/2

— Convertir des chaînes de caractères en nombres entiers ordinaux

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder()
df[["Sex", "Season"]] = enc.fit_transform(
    df.loc[:, ["Sex", "Season"]])
```

```
from sklearn.preprocessing import LabelEncoder
enc = LabelEncoder()
df[["Sex", "Season"]] = enc.fit_transform(
df.loc[:, ["Sex", "Season"]])
df["Sex"] = enc.fit_transform(df.loc[:, "Sex"])
```

Série 1D

	Sex	Season		Sex	Season
0	M	Summer	0	1.0	2.0
1	M	Summer	1	1.0	2.0
2	M	Summer	2	1.0	2.0
3	M	Summer	3	1.0	2.0
4	F	Springer	4	0.0	1.0
5	F	Autumn	5	0.0	0.0
6	F	Autumn	6	0.0	0.0
7	F	Springer	7	0.0	1.0
8	F	Winter	8	0.0	3.0
9	F	Winter	9	0.0	3.0
10	M	Summer	10	1.0	2.0
11	M	Autumn	11	1.0	0.0
12	M	Springer	12	1.0	1.0

Transformation des données *discrètes (Encoding)* 2/3

OrdinalEncoder / LabelEncoder 2/2

Comment spécifie-t-on l'ordre des catégories lors de l'utilisation de OrdinalEncoder ou LabelEncoder?!

```
from sklearn.preprocessing import OrdinalEncoder,  
LabelEncoder
```

```
ord_categories= ["M", "F"],  
               ["Summer", "Springer", "Autumn", "Winter"]]  
enc1 = OrdinalEncoder(categories=ord_categories)
```

```
enc2 = LabelEncoder(categories=["M", "F"])
```

```
df[["Sex", "Season"]] = enc1.fit_transform(  
    df.loc[:, ["Sex", "Season"]])
```

	Sex	Season
0	M	Summer
1	M	Summer
2	M	Summer
3	M	Summer
4	F	Springer
5	F	Autumn
6	F	Autumn
7	F	Springer
8	F	Winter
9	F	Winter
10	M	Summer
11	M	Autumn
12	M	Springer

	Sex	Season
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	1.0	1.0
5	1.0	2.0
6	1.0	2.0
7	1.0	1.0
8	1.0	3.0
9	1.0	3.0
10	0.0	0.0
11	0.0	2.0
12	0.0	1.0



Transformation des données *discrètes (Encoding)* 3/3

☰ OneHotEncoder/ get_dummies()

— Transformer l'espace de **n** descripteurs à **k** descripteurs binaires (**k** > **n**)

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False,
                    categories=[["M", "F"]])
df[["S=M", "S=F"]] = enc.fit_transform(
    df.loc[:, ["Sex"]])
    DataFrame 2D!
df.drop(labels=["Sex"], axis=1, inplace=True)
```

```
df=pd.get_dummies(df, columns=['Sex'],
dtype='int32')
```

	Sex	Season
0	M	Summer
1	M	Summer
2	M	Summer
3	M	Summer
4	F	Springer
5	F	Autumn
6	F	Autumn
7	F	Springer
8	F	Winter
9	F	Winter
10	M	Summer
11	M	Autumn
12	M	Springer

	Season	S=M	S=F
0	Summer	1.0	0.0
1	Summer	1.0	0.0
2	Summer	1.0	0.0
3	Summer	1.0	0.0
4	Springer	0.0	1.0
5	Autumn	0.0	1.0
6	Autumn	0.0	1.0
7	Springer	0.0	1.0
8	Winter	0.0	1.0
9	Winter	0.0	1.0
10	Summer	1.0	0.0
11	Autumn	1.0	0.0
12	Springer	1.0	0.0



Discrétisation des données **Méthodes**



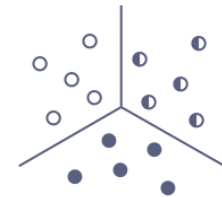
Equal-Width Discretization

'uniform'



Equal-Frequency Discretization

'quantile'



K-Means Discretization

'K-Means'

NB.

- Il n'y a pas une discrétisation optimale
- Il y a une démarche cohérente par rapport à un objectif donné et une distribution donnée

Discrétisation des données 1/2

Equal-Width Discretization 'uniform'

— Séparer toutes les valeurs possibles de chaque descripteur en **n_bins** intervalles (classes), chacune ayant la même largeur

$$\text{Width} = \frac{\max(x_i) - \min(x_i)}{n_bins}$$

```
from sklearn.preprocessing import KBinsDiscretizer
enc = KBinsDiscretizer(n_bins=4, encode='ordinal',
                       strategy='uniform', subsample=None)
df['Age'] = enc.fit_transform(df[['Age']])
```

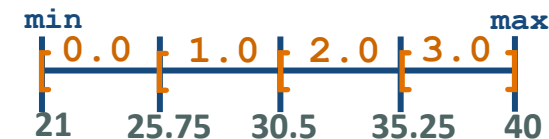
Age	
0	24.0
1	23.0
2	24.0
3	34.0
4	21.0
5	22.0
6	25.0
7	40.0
8	27.0
9	38.0
10	31.0

Age	
0	0.0
1	0.0
2	0.0
3	2.0
4	0.0
5	0.0
6	0.0
7	3.0
8	1.0
9	3.0
10	2.0

$$\frac{40-21}{4}=4.75$$

😊 Optimum pour distribution uniforme

😞 Inadaptée pour les distributions asymétriques



Discrétisation des données 2/2

Equal-Frequency Discretization 'quantile'

- Séparer toutes les valeurs possibles de chaque descripteur en **n_bins** intervalles (classes), chacune ayant $\pm$ le même nombre d'observation

```
from sklearn.preprocessing import KBinsDiscretizer  
enc = KBinsDiscretizer(n_bins=4, encode='ordinal',  
                        strategy='quantile', subsample=len(df))  
df['Age'] = enc.fit_transform(df[['Age']])
```

	Age		Age
0	24.0	0	1.0
1	23.0	1	0.0
2	24.0	2	1.0
3	34.0	3	3.0
4	21.0	4	0.0
5	22.0	5	0.0
6	25.0	6	2.0
7	40.0	7	3.0
8	27.0	8	2.0
9	38.0	9	3.0
10	31.0	10	2.0

21	22	23	24	24	25	27	31	34	38	40
0	1	2	3	4	5	6	7	8	9	10



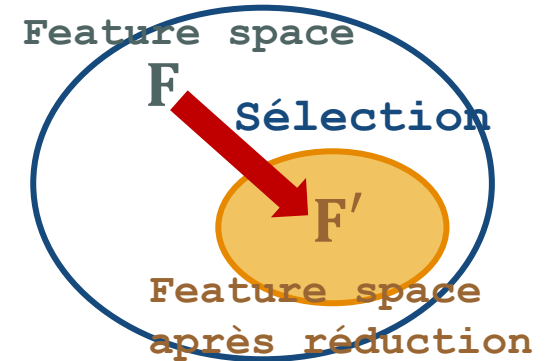
$$\text{Index}_i = \frac{i \times n}{n_bins} \quad \text{Index}_1 = \frac{1 \times 11}{4} = 2.75 \cong 3 \quad \text{Index}_2 = \frac{2 \times 11}{4} = 5.5 \cong 6 \quad \text{Index}_3 = \frac{3 \times 11}{4} = 8.25 \cong 9$$

☺ Optimum pour distribution symétrique et asymétrique



Sélection de descripteurs

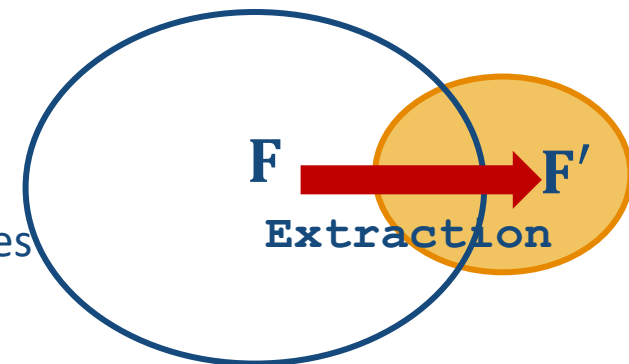
— Sélection des descripteurs les plus pertinentes à partir de l'ensemble initial de caractéristiques



Extraction de descripteurs

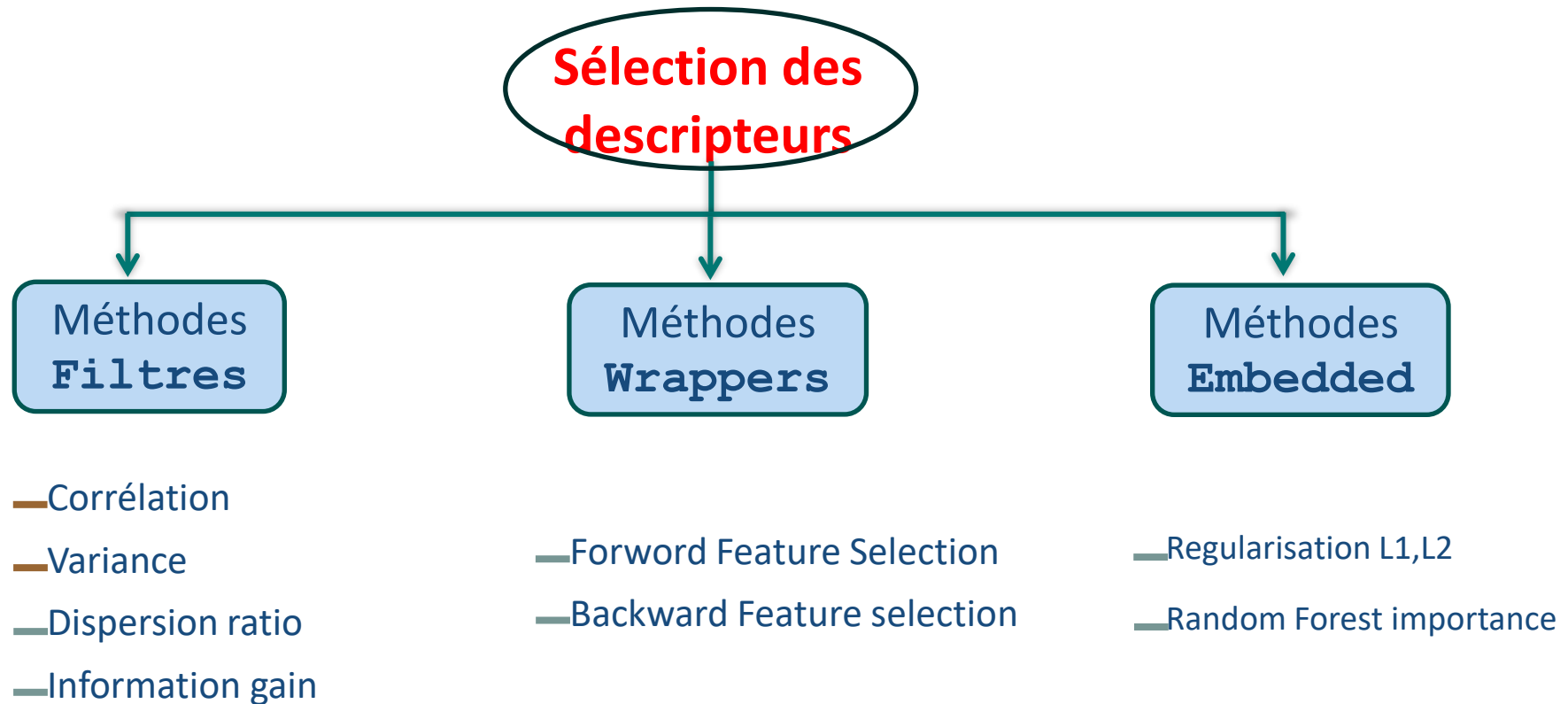
— Réduction basée sur une transformation des données

— Construire un nouveau ensemble réduit de descripteurs à partir de l'ensemble initial



F : feature space

F' : feature space après réduction



Analyse de corrélation 1/2



corrélation de Pearson=

mesure la relation linéaire entre deux variables

$$\text{Corr}(X, Y) = \frac{\sum_{i=1}^N (x_i - \bar{X})(y_i - \bar{Y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{X})^2 \sum_{i=1}^N (y_i - \bar{Y})^2}} \quad [-1..1]$$

$$\bar{X} = \frac{\sum_{i=1}^N x_i}{N} = \text{moyenne de } X$$

$$\bar{Y} = \frac{\sum_{i=1}^N y_i}{N} = \text{moyenne de } Y$$

avec N= Nombre d'observations

par défaut `numeric_only=False`

`df_corr=df.corr(numeric_only=True)`

Inclure uniquement des données de type float, int ou booléen!

`sns.heatmap(df_corr, annot=True, vmin=-1, vmax=1)`

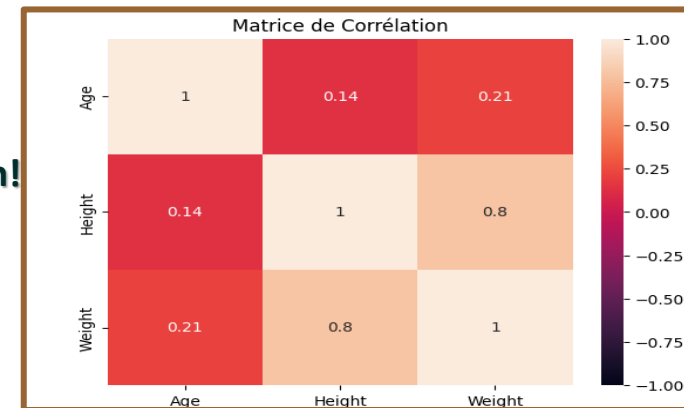
`plt.title('Matrice de Corrélation')`

`plt.show()`

X	Y
x_1	y_1
$\vdots$	$\vdots$
x_i	y_i
$\vdots$	$\vdots$
x_n	y_n

df=	Age	Height	Weight
0	24.0	180.0	80.0
1	23.0	170.0	60.0
2	24.0	NaN	NaN
3	34.0	NaN	NaN
4	21.0	185.0	82.0
...	...	...	...
271111	29.0	179.0	89.0
271112	27.0	176.0	59.0
271113	27.0	176.0	59.0
271114	30.0	185.0	96.0
271115	34.0	185.0	96.0

df	corr=	Age	Height	Weight
	Age	1.000000	0.138246	0.212069
	Height	0.138246	1.000000	0.796213
	Weight	0.212069	0.796213	1.000000

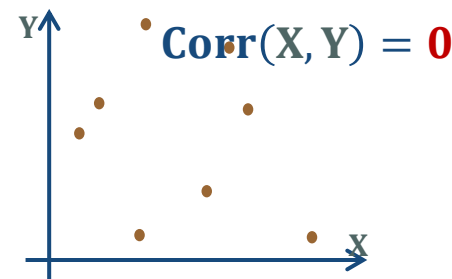
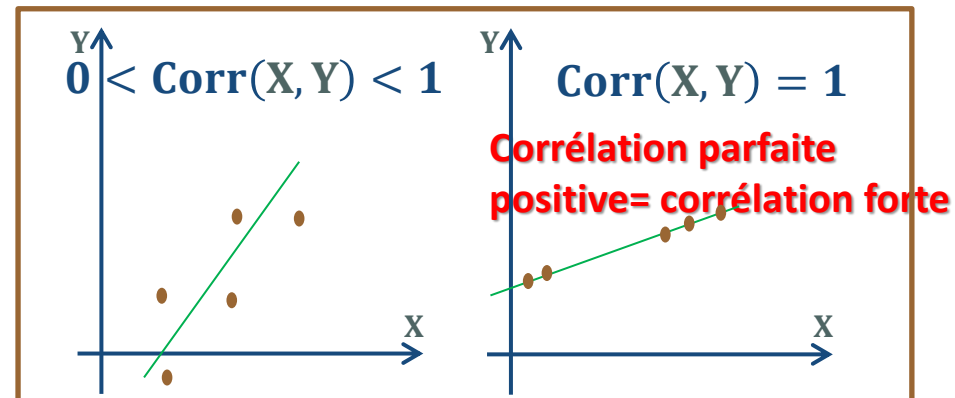
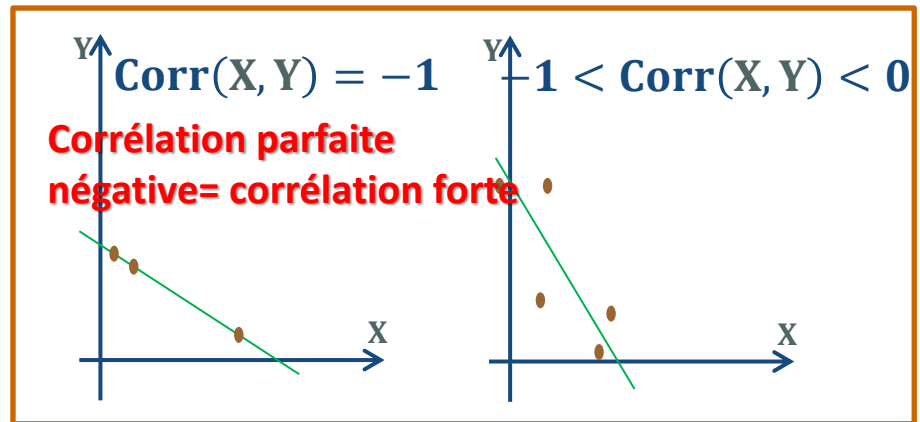


Analyse de corrélation 2/2

✓ **Corrélation négative** = X et Y évoluent ensemble de manière linéaire dans des directions opposées!

✓ **Corrélation Positive** = X et Y évoluent ensemble de manière linéaire dans la même direction!

✓ **Corrélation=0** = Absence de corrélation linéaire entre les attributs X et Y !



VarianceThreshold

$$\text{var}(X_j) = \frac{1}{N} \sum_{i=1}^N (a_{ij} - \bar{X}_j)^2$$

$[0, +\infty[$

$$\bar{X}_j = \frac{1}{N} \sum_{i=1}^N a_{ij}$$

	X_j
obs1	a_{1j}
$\vdots$	$\vdots$
obsi	a_{ij}
obsN	a_{Nj}

Tous les attributs doivent être de type numérique + Pas de NaN



df=	Sex	Age	Height	Weight	Year	City	Sport
0	M	24.0	180.0	80.0	1992	Barcelona	Basketball
1	M	23.0	170.0	60.0	2012	London	Judo
4	F	21.0	185.0	82.0	1988	Calgary	Speed Skating
5	F	21.0	185.0	82.0	1988	Calgary	Speed Skating
6	F	25.0	185.0	82.0	1992	Albertville	Speed Skating
...	...	...	...	...	...	...	...
271111	M	29.0	179.0	89.0	1976	Innsbruck	Luge
271112	M	27.0	176.0	59.0	2014	Sochi	Ski Jumping
271113	M	27.0	176.0	59.0	2014	Sochi	Ski Jumping
271114	M	30.0	185.0	96.0	1998	Nagano	Bobsleigh
271115	M	34.0	185.0	96.0	2002	Salt Lake City	Bobsleigh

```
from sklearn.feature_selection import VarianceThreshold
thresholder = VarianceThreshold(threshold=40)
```

Par défaut = 0

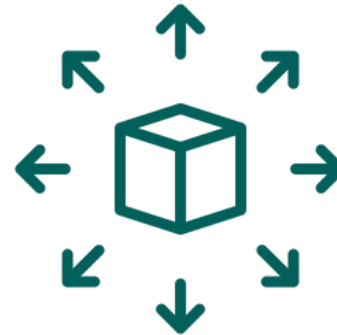
```
data_high_variance = thresholder.fit_transform(
    df.loc[:,["Age","Height","Weight","Year"]])
```

↑
ndarray comportant les attributs ayant une variance $\geq$ threshold

Réduction des données **Extraction des descripteurs**



Méthodes linéaires



Méthodes non-linéaires



CHAPITRE 3

APPRENTISSAGE SUPERVISÉ : ESTIMATION DES PERFORMANCES



Processus à **deux phases** :

- **Apprentissage (off-line)** : construire un modèle $\mathcal{M}$ qui décrit un ensemble prédéterminé de classes de données
NB. La construction du modèle doit être basé sur des données **annotés (étiquetés)**
- **Test (on-line)** : utiliser le modèle obtenu $\mathcal{M}$ pour affecter une classe à une nouvelle observation

Classification & Régression : Apprentissage supervisé

Jeu de données d'apprentissage

Classe désirée

	outlook	temperature	humidity	windy	play
0	sunny	hot	high	False	no
1	sunny	hot	high	True	no
2	overcast	hot	high	False	yes
3	rainy	mild	high	False	yes
4	rainy	cool	normal	False	yes
5	rainy	cool	normal	True	no
6	overcast	cool	normal	True	yes
7	sunny	mild	high	False	no
8	sunny	cool	normal	False	yes
9	rainy	mild	normal	False	yes

Apprentissage supervisé

Est-ce-que le modèle obtenu est performant?

Modèle $\mathcal{M}$

Nouvelle observation

Classe à prédire

outlook	temperature	humidity	windy	play
sunny	mild	normal	True	?



Spécifier des données de validation : Echantillonnage



Estimer la performance du modèle : Validation



Base de données
(Dataset)



Ensemble d'apprentissage
(Training set)



Ensemble de Test
(Test set)

Méthodes d'échantillonnage



Split validation

Processus d'apprentissage et de test n'est effectué **qu'une seule fois**



Cross-validation

Faire **plusieurs tests** sur différents ensembles d'apprentissage et de test



Split validation

- Processus de validation qui s'exécute qu'une seule fois
- Diviser le jeu de données en **deux groupes** :
 - Ensemble d'apprentissage** : utilisé pour former le modèle
 - Ensemble de test** : utilisé pour estimer la performance du modèle formé

Ensemble d'apprentissage (70%)

Ensemble de test (30%)

Cross validation : K-Folds

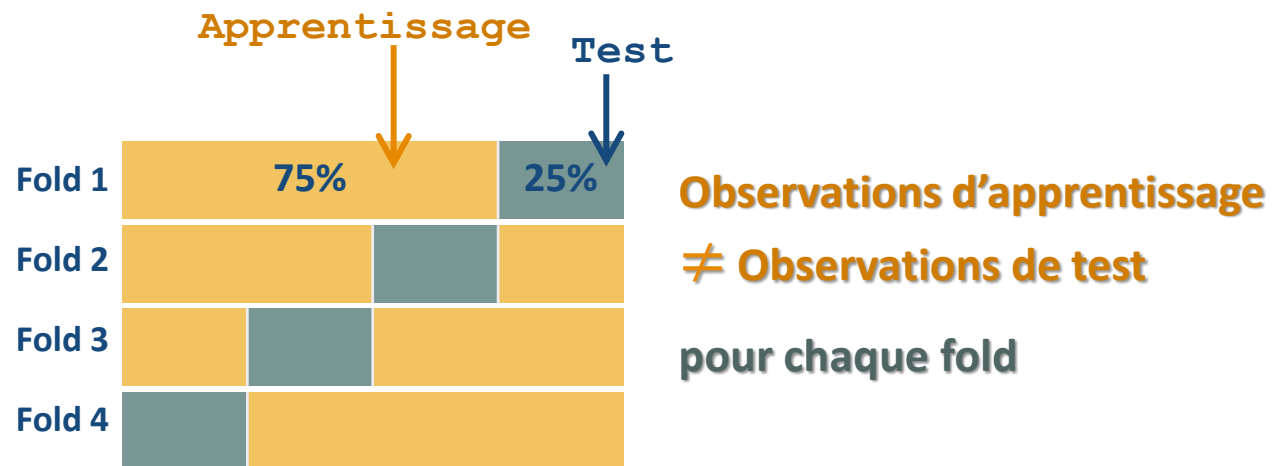
- Diviser **k** fois le jeu de données de **N** observations

Pour chaque fold : **1.. k**

- Sélectionner $N \cdot \frac{k-1}{k}$ observations comme **ensemble d'apprentissage**
- Les restes des observations comme **ensemble de test**

NB. Répéter l'opération de sorte que toutes les observations aient été utilisé exactement une fois pour le test

Exemple k= 4



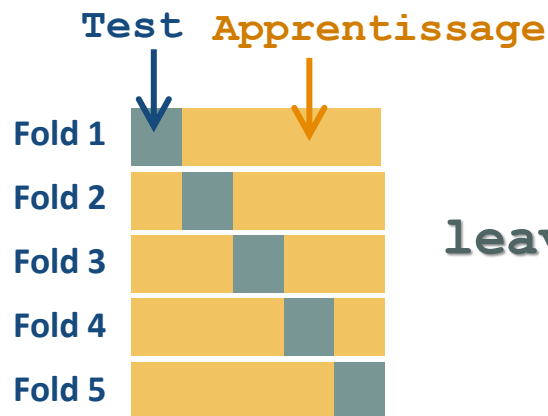
Cross validation : leave-one-out

- Diviser N fois le jeu de données de N observations

Pour chaque fold : $1.. N$

- Sélectionner une observation pour le test
- Les restes $N-1$ observations comme ensemble d'apprentissage

Exemple $k=5$



leave-one-out = k-fold avec $k= N$

Métriques d'évaluation pour la classification (1/3)

Matrice de confusion (avec N classes)

Tableau **bidimensionnel** N lignes × N colonnes

CM(i,i): Nombre d'observations désirées comme C_i et prédites comme C_i

N : le nombre total des observations

CM		Classe prédite			
		C_1	...	C_i	C_N
Classe désirée	C_1	CM(1,1)		CM(1,i)	
	$\vdots$		$\vdots$		
	C_i			CM(i,j)	
	C_N				CM(N,N)

—Taux de classification **Accuracy** τ

$$\tau = \frac{\sum_{i=1}^N CM(i,i)}{n}$$

—Taux d'erreur **Error rate** e

$$e = \frac{\sum_{i=1}^N \sum_{j=1}^N CM(i,j)}{n} = 1 - \tau$$

	A1	A2	...	C.Désirée	C.Prédite
1				Oui	Oui
2				Non	Oui
3				Non	Non
4				Non	Oui
5				Oui	Non
6				Oui	Oui
7				Non	Non
8				Oui	Oui

Métriques d'évaluation pour la classification (2/3)

- **Rappel** recall_{C_i} = Mesure la proportion d'observations correctement prédites comme i parmi tous les observations désirées comme i

$$\text{recall}_{C_i} = \frac{\text{CM}(i,i)}{\sum_{j=1}^N \text{CM}(i,j)}$$

CM		Classe prédite			
		C_1	...	C_i	C_N
Classe désirée	C_1	CM(1,1)		CM(1,i)	
	$\vdots$		$\vdots$		
	C_i			CM(i,j)	
	C_N				CM(N,N)

Cas de 2 classes **yes**, **no**

$$\begin{aligned} \text{recall}_{\text{yes}} &= \text{True Positive rate} \\ &= \text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}} \end{aligned}$$

$$\begin{aligned} \text{recall}_{\text{no}} &= \text{True Negative rate} \\ &= \text{Specificity} = \frac{\text{TN}}{\text{FP} + \text{TN}} \end{aligned}$$

		Classe prédite	
		yes	no
Classe désirée	yes	TP	FN
	no	FP	TN

True Positive (arrow to TP)
 False Negative (arrow to FN)
 False Positive (arrow to FP)
 True Negative (arrow to TN)

Métriques d'évaluation pour la classification (3/3)

— **Précision** precision_{C_i} = Mesure la proportion d'observations correctement prédites comme i parmi tous les observations prédites comme i

$$\text{precision}_{C_i} = \frac{\text{CM}(i,i)}{\sum_{j=1}^N \text{CM}(j,i)}$$

CM		Classe prédite			
		C_1	...	C_i	C_N
Classe désirée	C_1	CM(1,1)		CM(1,i)	
	$\vdots$		$\vdots$		
	C_i			CM(i,i)	
	C_N				CM(N,N)

— **F- measure** F1 - score_{C_i} = Mesure la moyenne harmonique de rappel et précision

$$\text{F1 - score}_{C_i} = \frac{2 \times (\text{precision}_{C_i} \times \text{recall}_{C_i})}{\text{precision}_{C_i} + \text{recall}_{C_i}}$$

la précision et le rappel sont pondérés de façon égale

Exemple

Étant donné le modèle de prédiction Γ qui permet de déterminer si l'objet dans une image est un **Chat**

200 images sont utilisées pour tester le performance de Γ : 170 contiennent des chats, et 30 ne contiennent pas de chats

Le résultat de prédiction du modèle Γ est 160 contiennent des chats, et 40 ne contiennent pas de chats (140 uniquement désirées comme **Chat** et prédites comme **Chat**)

Déterminer la matrice de confusion associée à Γ .

Déduire l'accuracy, la spécificité, et la précision de la classe Chat ?

$$\text{accuracy} = \frac{140 + 10}{200} = 75\%$$

$$\text{recall}_{\text{Chat}} = \text{sepcificity} = \frac{10}{20 + 10} = 33.3\%$$

$$\text{precision}_{\text{Chat}} = \frac{140}{140 + 20} = 87.5\%$$

		Classe prédite	
		Chat	$\overline{\text{Chat}}$
Classe désirée	Chat	140	30
	$\overline{\text{Chat}}$	20	10

Quiz

1. Étant donnée un jeu de données de 20 observations; le taux de classification obtenu en appliquant la validation croisée **20-Folds** est égal à **91.6%** ; En appliquant **leave-one-out**, le taux d'erreur est égal à (seule réponse)

- ☒ 8.4%
- ☐ 91.6%
- ☐ 50%
- ☐ Aucune bonne réponse

2. Soit la matrice de confusion suivante : (plusieurs réponses)

	C1	C2	C3
C1	10	1	3
C2	0	8	0
C3	2	2	11

- ☒ Le nombre total d'observations est égal à 37
- ☒ Le nombre d'observations mal classées est égal à 8
- ☐ $\text{recall}_{C2} = 0.73$
- ☐ $\text{precision}_{C2} = 1$

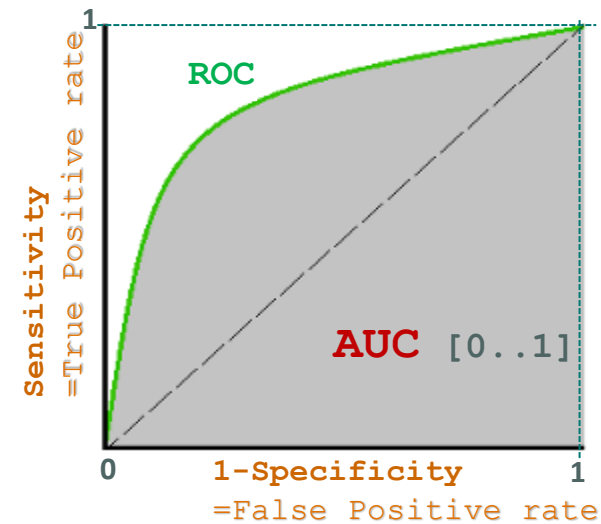
ROC/ AUC (Classification Binaire)

✓ Courbe Receiver Operating Characteristics (ROC)

- Permet de visualiser les performances d'un modèle de **classification binaire** en fonction de différents seuils de décision
- Evalue la capacité d'un modèle à séparer les classes positives et négatives, indépendamment du seuil choisi
- Permet de comparer différents modèles pour voir lequel a la meilleure capacité de discrimination
- Insensible aux classes déséquilibrées

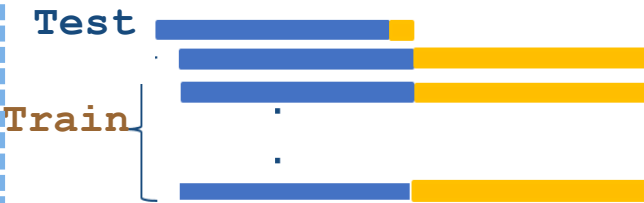
✓ Area Under Curve(AUC)

- Surface sous la courbe ROC : résume en une seule valeur la performance d'un modèle, peu importe le seuil



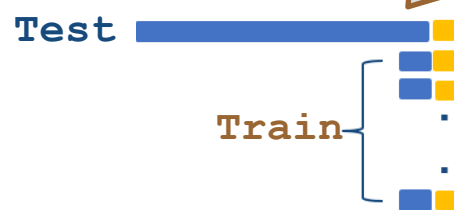
- **AUC= 1.0** : Le modèle est parfait (distingue parfaitement les classes)
- **AUC= 0.5** : Le modèle ne fait pas mieux qu'un classificateur aléatoire
- **AUC< 0.5** : Le modèle est pire qu'un classificateur aléatoire (inverse les classes)

Unbalanced training sets!



**Sur échantillonnage
(Over-sampling)**

Augmenter la fréquence des instances de la classe minoritaire en ajoutant des duplicatas ou en générant des données synthétiques



**Sous échantillonnage
(Under-sampling)**

Réduire la fréquence des instances de la classe majoritaire en éliminant certaines instances



**Échantillonnage hybride
(Hybrid sampling)**

Combine la suréchantillonnage avec la sous-échantillonnage



- SMOTE (Synthetic Minority Over-sampling Technique)
- ADASYN (Adaptive Synthetic Sampling)

Métriques d'évaluation pour la régression (1/2)

- **Mean Absolute Error (MAE)**= moyenne des valeurs absolues des erreurs entre les prédictions et les valeurs réelles

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad [0, +\infty[$$

- **Mean Squared Error (MSE)**= moyenne des carrés des erreurs entre les prédictions et les valeurs réelles

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad [0, +\infty[$$

- **Coefficient de détermination (R2)**= indique dans quelle mesure la variance des valeurs prédites par un modèle correspond à la variance réelle des données

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad [0, 1]$$

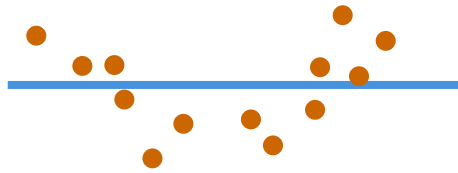
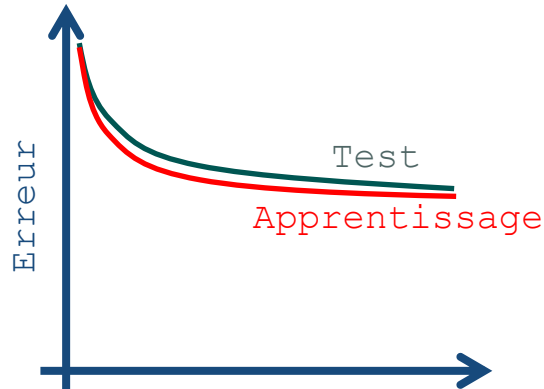
$$\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

	A1	A2	...	C.Désirée y	C.Prédite $\hat{y}$
1				60	70
2				70	90
3				80	110
4				90	130

N : le nombre total des observations

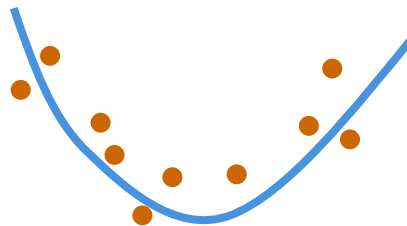
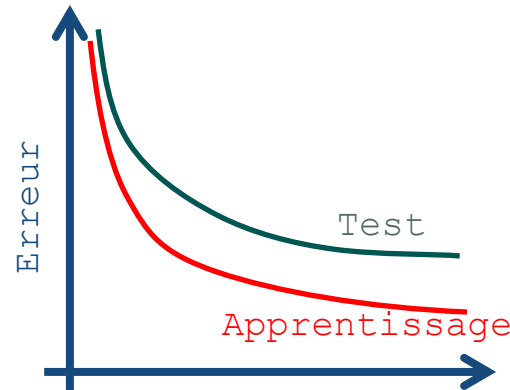
MSE donne plus de poids aux erreurs plus importantes!

Sous-apprentissage vs. Sur-apprentissage

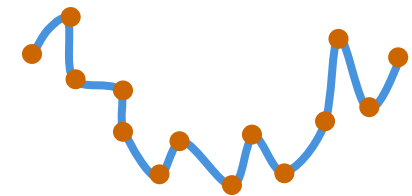
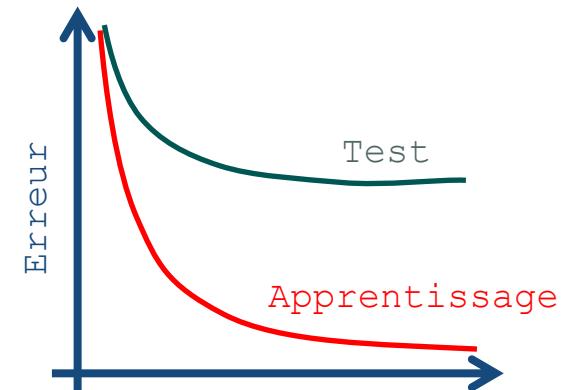


Sous-apprentissage
Underfitting

Incapacité à apprendre
les caractéristiques



Parfait
perfect



Sur-apprentissage
Overfitting

Learning noise



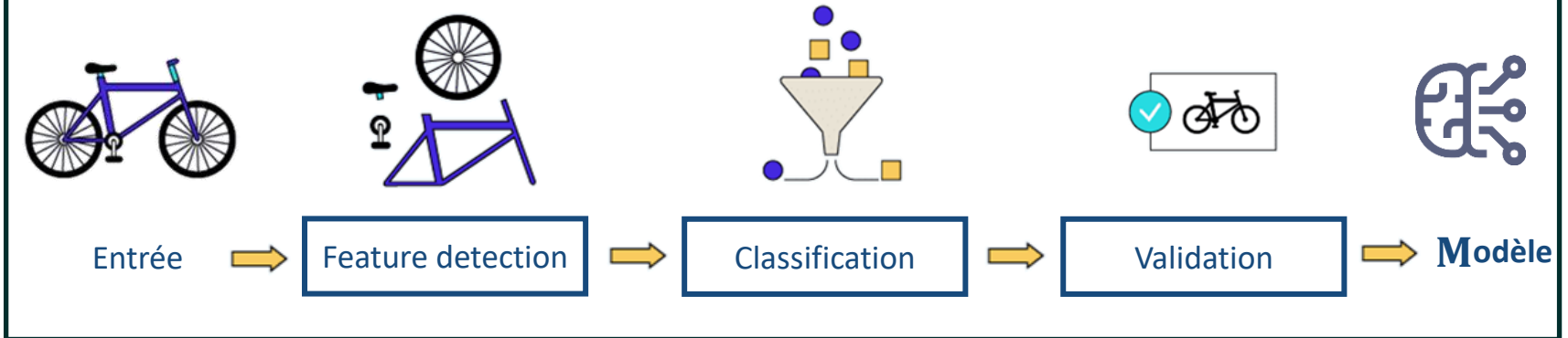
CHAPITRE 4

RÉSEAUX DE NEURONES : DE MACHINE LEARNING VERS LE DEEP LEARNING

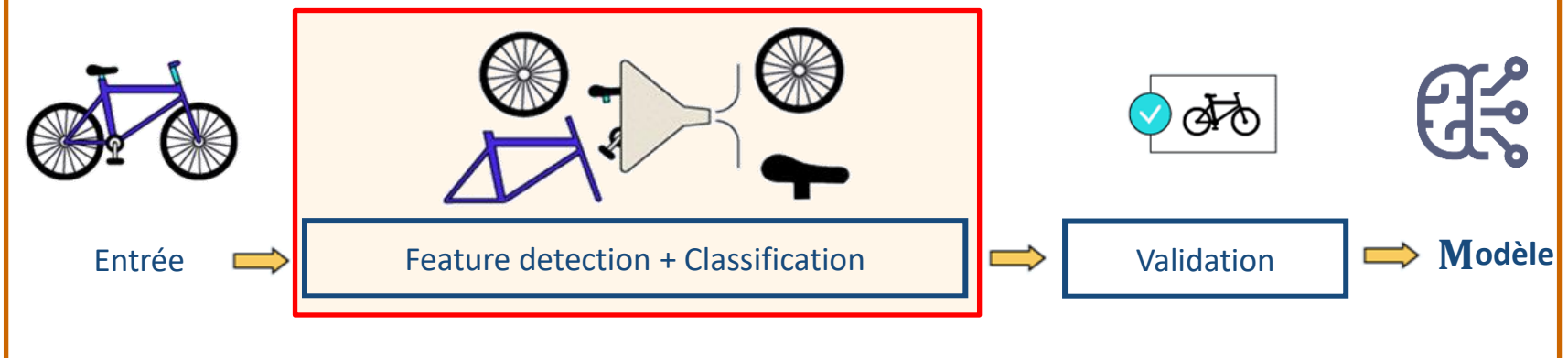


Apprentissage automatique vs. Apprentissage profond 1/2

Apprentissage automatique (ML)



Apprentissage profond (DL)



Apprentissage automatique vs. Apprentissage profond 2/2

	Apprentissage automatique (ML)	Apprentissage profond (DL)
Matériel (puissance de calcul)	CPU (Central Processing Unit)	GPU (Graphics Processing Unit))
Données	Des milliers d'observations	Big Data : des millions d'observation
Descripteurs	Handcrafted (bas niveau)	High-level (haut niveau)
Modélisation	Niveau par niveau	Bout en bout
Domaines d'applications	Classification & régression sur des données tabulaires	Classification & régression sur des données tabulaires, vision par ordinateur, NLP et audio

Domaines d'application!

- ✓ Vision par ordinateur & reconnaissance d'images
- ✓ Traitement du langage naturel (NLP)
- ✓ Reconnaissance vocale



Quelques applications ✓



Vision par ordinateur & reconnaissance d'images

- Reconnaissance d'objets
- Reconnaissance d'actions
- Reconnaissance des visages et des émotions
- Segmentation d'images
- Ré identification des personnes



Traitement du langage naturel & Reconnaissance vocale

- Traduction automatique
- Systèmes de recommandation intelligents
- Génération automatique de texte
- Chatboot conversationnelle
- Speech recognition



Succès de Deep Learning



AlphaGo Zero
2017



Amazon Rekognition
2019



Google car
2020



Google Smart Displays
2021



Apple Live Text
2021



Open AI DALL-E2
2022



Open AI ChatGPT
2022



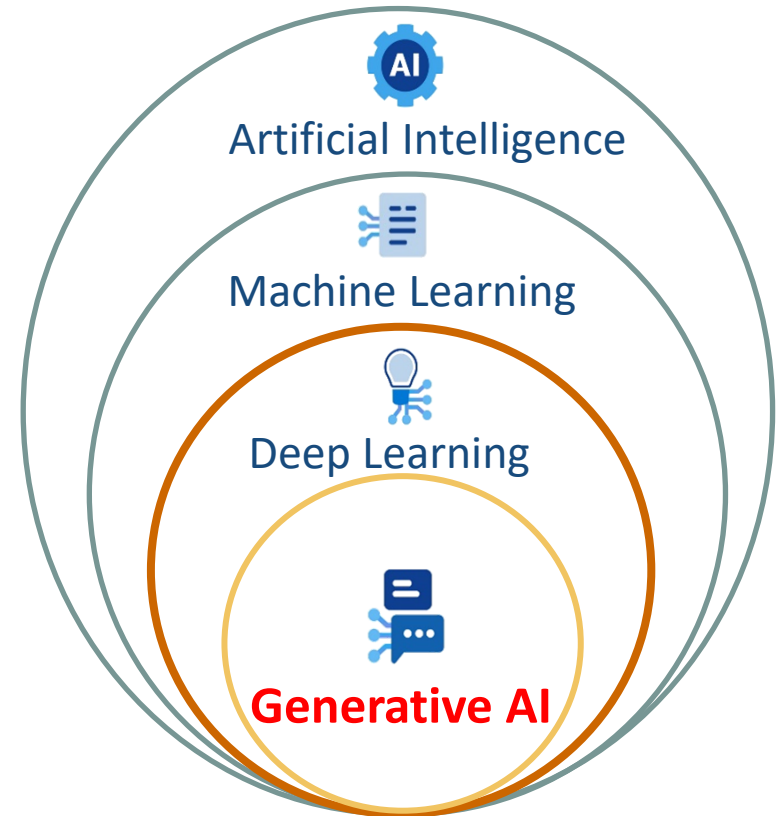
Google Bard
2023

IA générative

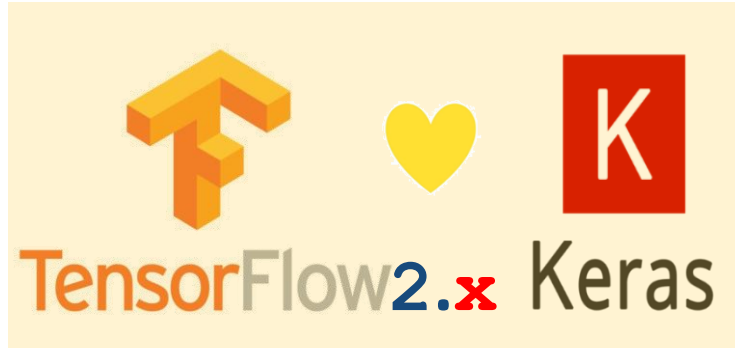
✓ Fait référence à des modèles qui sont capables de générer de nouvelles données à partir de données existantes

💡 Génération de texte, d'images et d'audio

- Variational Autoencoders
- Generative Adversarial Networks (GAN)
- Les modèles basés sur les Transformers (LMM, BERT, GPT, ViT, etc.)



Frameworks de DL




PyTorch

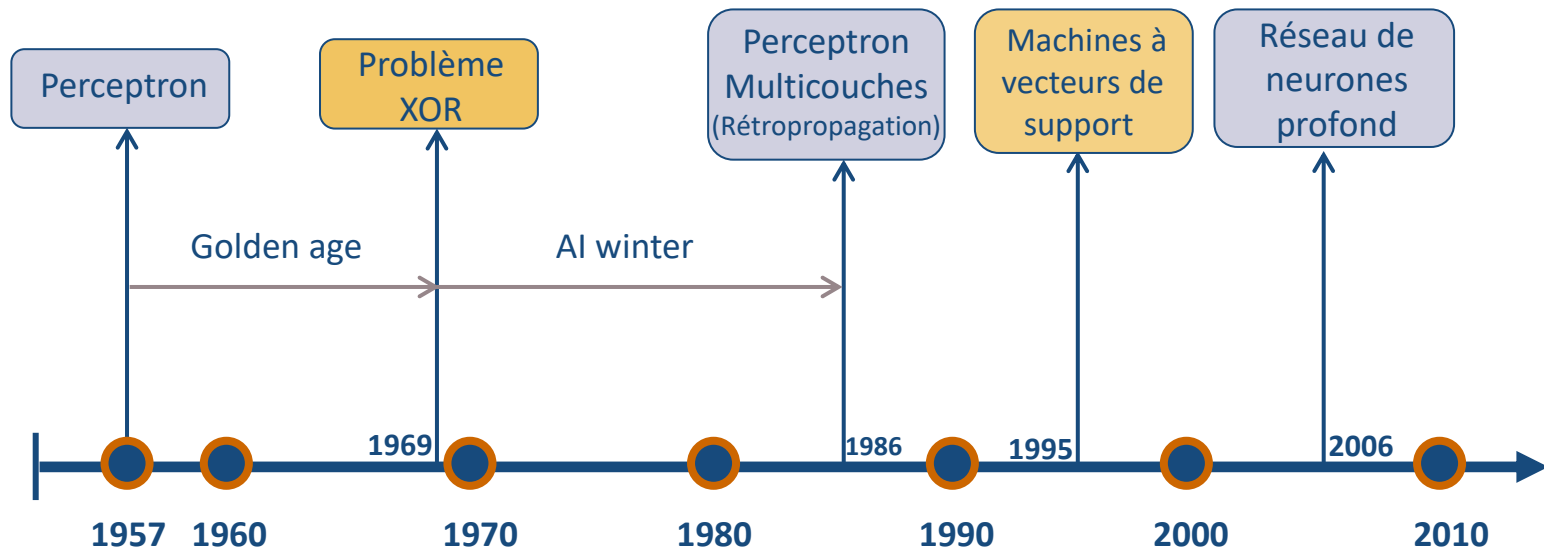
Caffe

mxnet

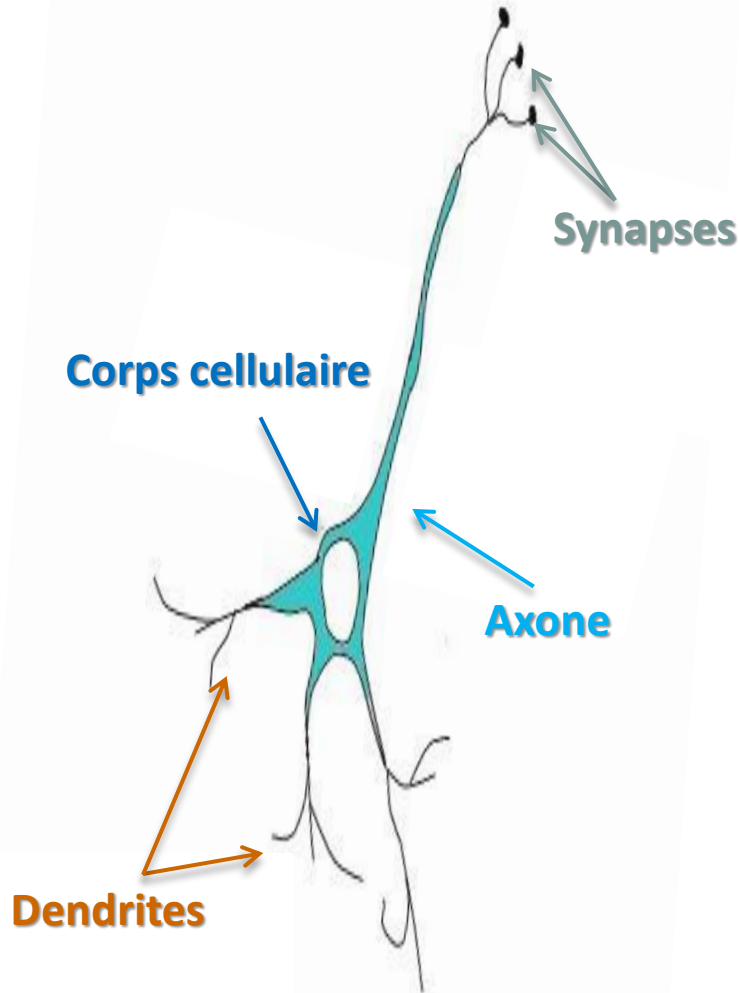
Quiz

1. L'apprentissage profond intègre à la fois la détection des descripteurs de haut niveau et la classification. Il nécessite une quantité de données massives et un GPU (seule réponse)
 - ☒ Vrai
 - ☐ Faux
2. TensorFlow 2.x (plusieurs réponses)
 - ☒ Intègre Keras, ce qui améliore considérablement la convivialité
 - ☒ Est un Framework Open Source
 - ☒ Est un Framework flexible
 - ☐ Est inventé par Facebook
3. Parmi les éléments suivants, lequel ne fait pas partie du Framework de développement de Deep Learning (seule réponse)
 - ☐ Caffe
 - ☐ Keras
 - ☒ skit-learn
 - ☐ MXNet

Historique du développement des réseaux de neurones



Neurone biologique



- Transmettent l'information venue de l'extérieur vers le corps cellulaire
- **=centre de contrôle** Fait la somme des informations qui lui parviennent
- Traite l'information et renvoie le résultat sous forme de signaux électriques, du corps cellulaire à l'entrée des autres neurones
- Permettent aux neurones de **communiquer** entre eux

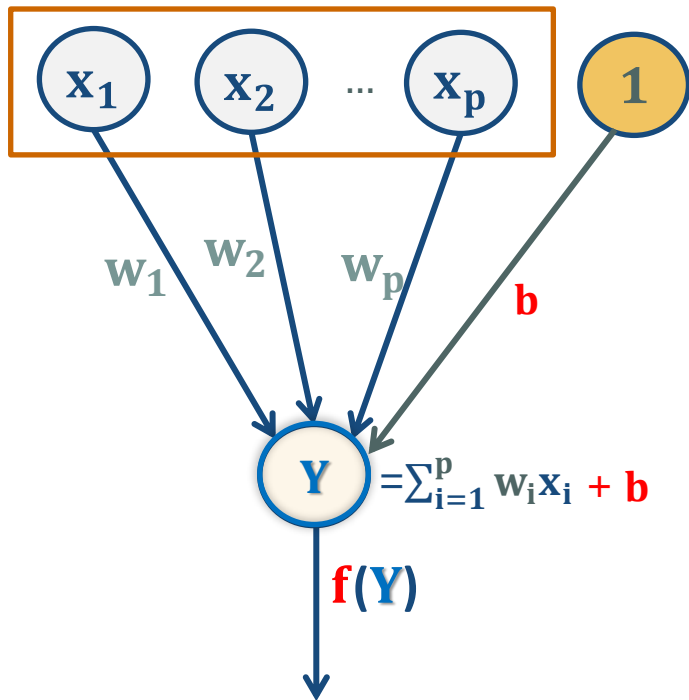
$\approx 10^{11}$ neurones dans le cerveau humain
 $\approx 10^4$ connexions (synapses + axones) / neurone



APPRENTISSAGE DES RÉSEAUX DE NEURONES

Perceptron simple

Neurone Artificiel **Perceptron simple** 1/2



x_1	x_2	...	x_p	Classe_d

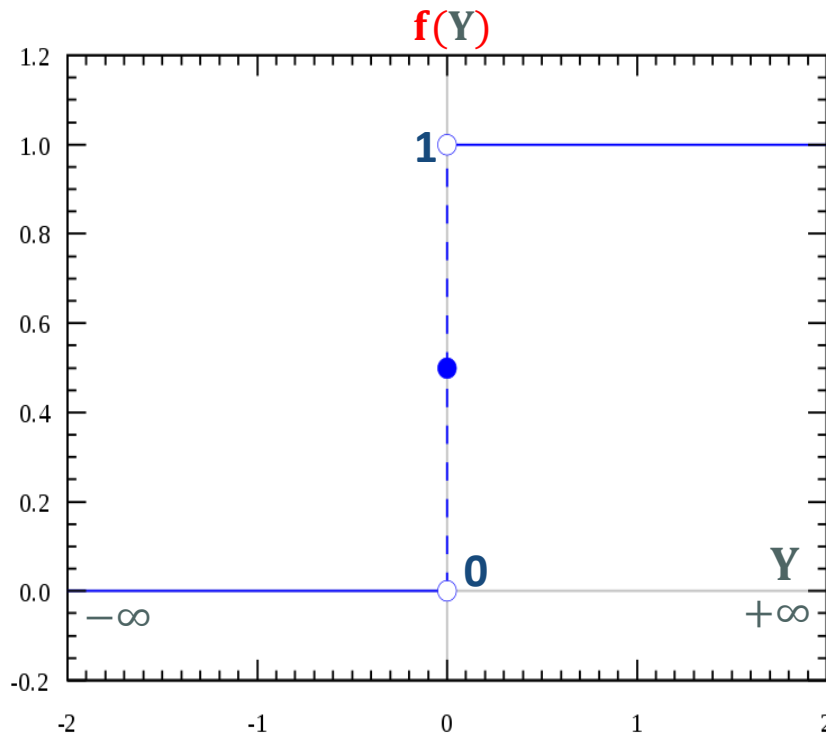
- Dendrites** : Série d'entrées $x_1 \dots x_p$
- Synapses** : coefficients synaptiques $w_1 \dots w_p$
=> Chaque entrée est associée à un poids
- Corps cellulaire** : Traitement (calcul de Y)
- Axone** : Fonction d'activation f
=> Résultat de neurones = sortie

Bias b = Régulateur
=> Aide à s'adapter au mieux aux données

Neurone Artificiel **Perceptron simple** 1/2



Fonction d'activation = Heaviside (Marche)



$$Y = \sum_{i=1}^p w_i x_i + b$$

$$f(Y) = 1 \text{ si } Y \geq 0$$

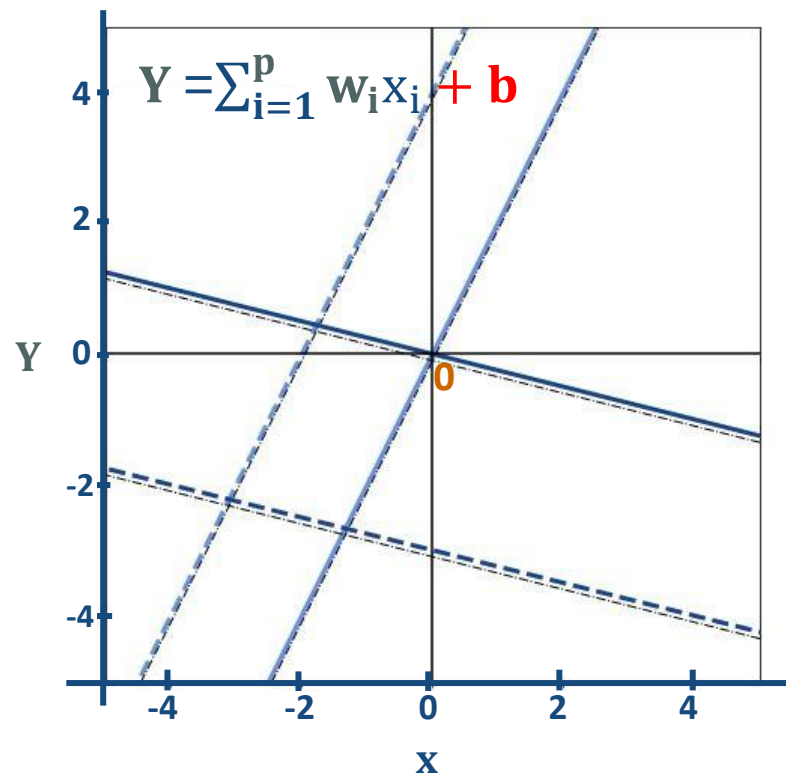
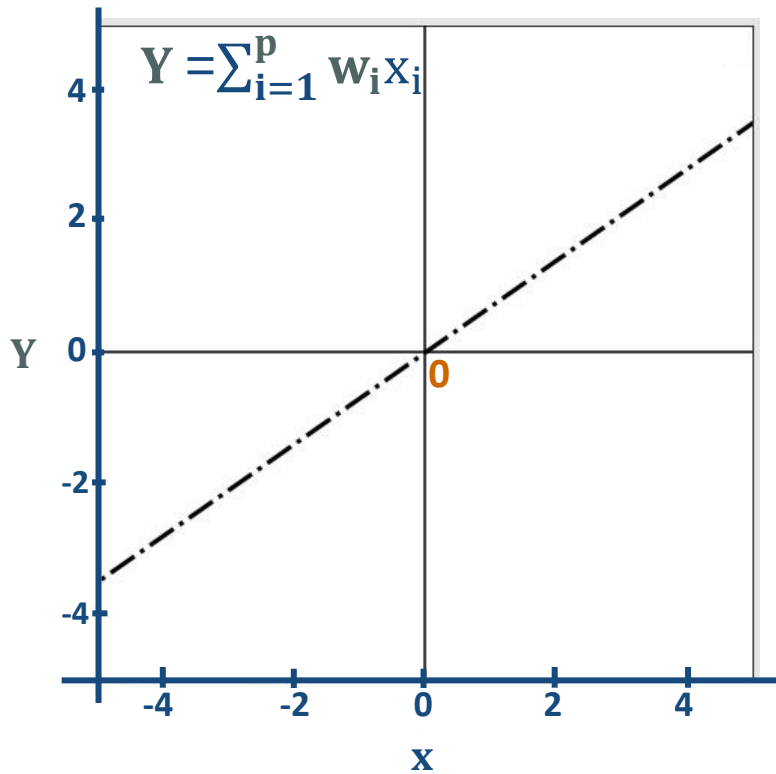
$$f(Y) = 0 \text{ sinon}$$

Deux valeurs : **0** ou **1**

☑ Classification **binaire**

☑ Problème linéairement séparable!

Bias !



Loi d'apprentissage = Ajustement des poids

✓ Loi de Widrow-Hoff (Delta rule)

$$w_i = w_i + \eta (d - \text{sortie}) x_i$$

Diagram illustrating the Delta rule equation with annotations:

- Erreur (loss)**: A bracket above the term $(d - \text{sortie})$ indicates the error or loss.
- Classe désirée**: An arrow points to d (the target class).
- Classe prédite = $f(Y)$** : An arrow points to sortie (the predicted class).
- η** : An arrow points to the learning rate symbol.

Pas d'apprentissage

= **Learning rate** $\in]0..1]$

Contrôle la rapidité avec laquelle le modèle est adapté au problème

Si **sortie**=**d**, w_i sont inchangables

Si **sortie**<**d**, w_i va diminuer

Si **sortie**>**d**, w_i va augmenter

Exemple introductif (1/4)

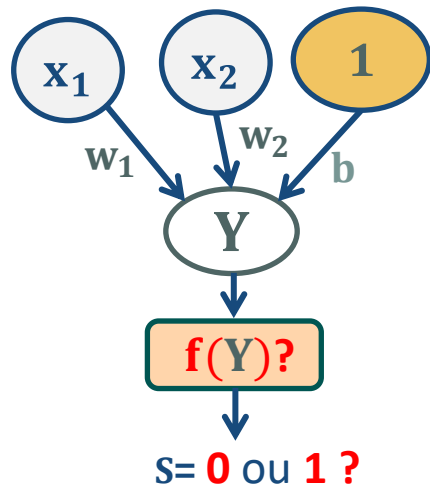
Étant donné le jeu de donnée d'apprentissage `or_logique`

	x_1	x_2	d
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

Variable à prédire (cible) : $d \rightarrow$ classification binaire (0/ 1)

Variables explicatives : x_1 , et x_2

* Représenter l'architecture du perceptron associé à `or_logique`?



Exemple introductif (2/4)

💡 Objectif : ajuster les poids w_1 , w_2 , et b !

❓ Etapes à suivre !

1- Initialiser aléatoirement les poids et Bias w_1 , w_2 , et b

$$w_1 = 1, w_2 = -1, b = 0$$

	w_1	w_2	b	x_1	x_2	d	Y	$f(Y)$	w_1	w_2	b
0	-	-	-	-	-	-	-	-	1	-1	0

2- Faire passer la première observation ($x_1 = 0$; $x_2 = 0$; $d=0$) sur le réseau et mettre à jour w_1 , w_2 , et b sachant que $\eta=1$

	w_1	w_2	b	x_1	x_2	d	Y	$f(Y)$	w_1	w_2	b
1	1	-1	0	0	0	0	0	1	1	-1	-1

$$Y = \sum_{i=1}^2 w_i x_i + b = w_1 x_1 + w_2 x_2 + b = (1 \times 0) + (-1 \times 0) + 0 = 0$$

$$f(Y) = 1 \text{ puisque } Y \geq 0$$

$$w_1 = w_1 + \eta(d - \text{sortie})x_1 = w_1 + \eta(d - f(Y))x_1 = 1 + 1(0 - 1)0 = 1$$

⋮

Exemple introductif (3/4)

3- Faire passer les trois observations restantes une par une sur le réseau et mettre à jour w_1 , w_2 , et b sachant que $\eta=1$

	w_1	w_2	b	x_1	x_2	d	Y	$f(Y)$	w_1	w_2	b
2	1	-1	-1	0	1	1	-2	0	1	0	0
3	1	0	0	1	0	1	1	1	1	0	0
4	1	0	0	1	1	1	1	1	1	0	0

4-

Répéter

Repasser les observations une par une sur le réseau et mettre à jour w_1 , w_2 , et b sachant que $\eta=1$

Jusqu'à aucune correction n'est effectuée en passant toutes les observations (convergence)

⋮

Exemple introductif (4/4)

⋮

	w_1	w_2	b	x_1	x_2	d	Y	$f(Y)$	w_1	w_2	b
0	-	-	-	-	-	-	-	-	1	-1	0
1	1	-1	0	0	0	0	0	1	1	-1	-1
2	1	-1	-1	0	1	1	-2	0	1	0	0
3	1	0	0	1	0	1	1	1	1	0	0
4	1	0	0	1	1	1	1	1	1	0	0
5	1	0	0	0	0	0	0	1	1	0	-1
6	1	0	-1	0	1	1	-1	0	1	1	0
7	1	1	0	1	0	1	1	1	1	1	0
8	1	1	0	1	1	1	2	1	1	1	0
9	1	1	0	0	0	0	0	0	1	1	-1
10	1	1	-1	0	1	1	0	1	1	1	-1
11	1	1	-1	1	0	1	0	1	1	1	-1
12	1	1	-1	1	1	1	1	1	1	1	-1
13	1	1	-1	0	0	0	-1	0	1	1	-1

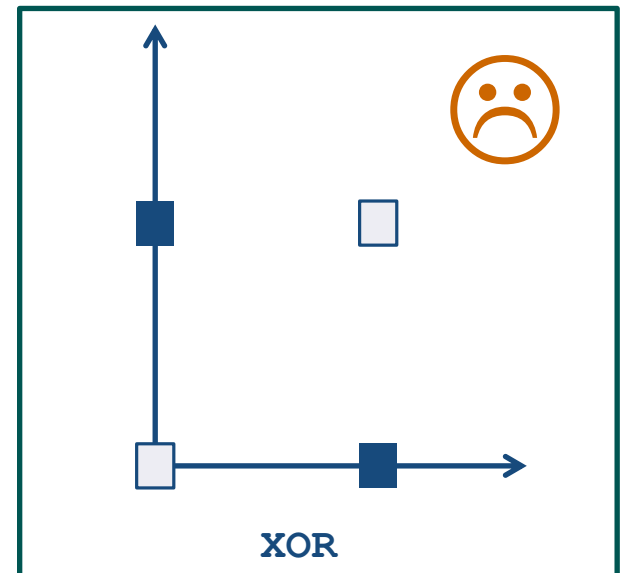
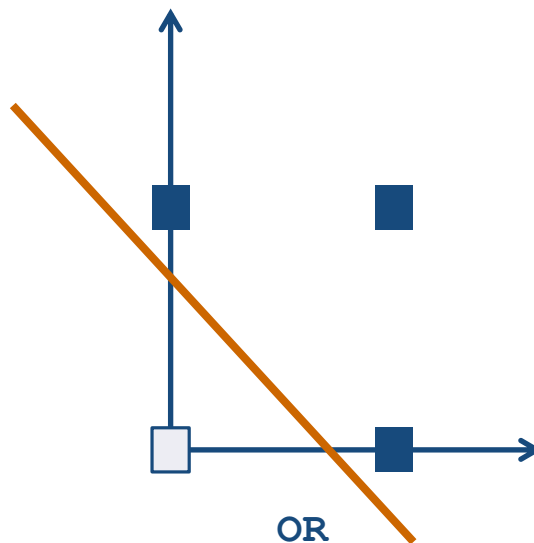
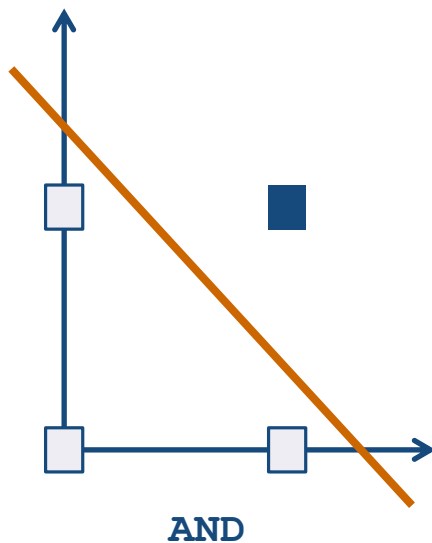
**ÉPOQUE
(EPOCH)**

Conditions d'arrêt

-  Les poids sont stables → aucune mise à jour n'effectuée en passant une époque
-  L'erreur globale ne diminue plus significativement
-  Le taux d'erreur atteint un minimum fixé (seuil)
-  Le nombre d'itérations atteint un maximum fixé
-  Le nombre d'époques est atteint

Perceptron & Problème XOR!

Incapable de classer les données non séparables linéairement



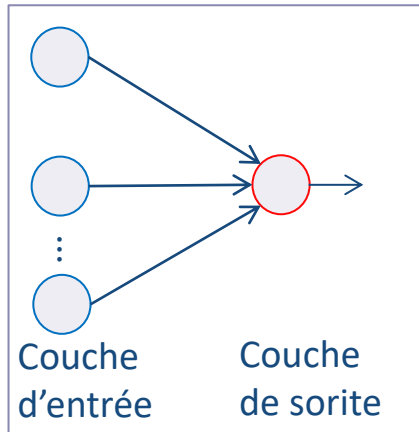
Quiz

1. Le problème XOR peut être résolu en utilisant un perceptron simple (seule réponse)
☐ Vrai
☒ Faux
2. Étant donnée l'observation $\mathbf{z} (4, 6, 7, 1)$ de 3 trois descripteurs et une classe (0 ou 1), le passage de cette observation sur un perceptron ne change pas les poids sachant que $\eta = \text{Tous les poids} = 1$ (seule réponse)
☒ Vrai
☐ Faux
3. Combien il existe de paramètres à apprendre au total à partir d'un perceptron simple entraîné sur un dataset comportant 200 observations et 20 descripteurs ? (seule réponse)
☐ 201
☐ 200
☒ 21
☐ 20

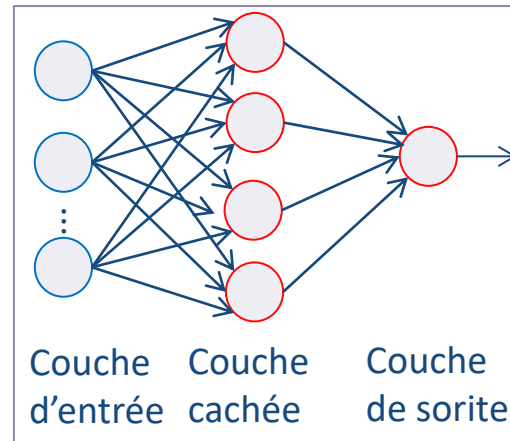
APPRENTISSAGE DES RÉSEAUX DE NEURONES

MLP: Fully connected

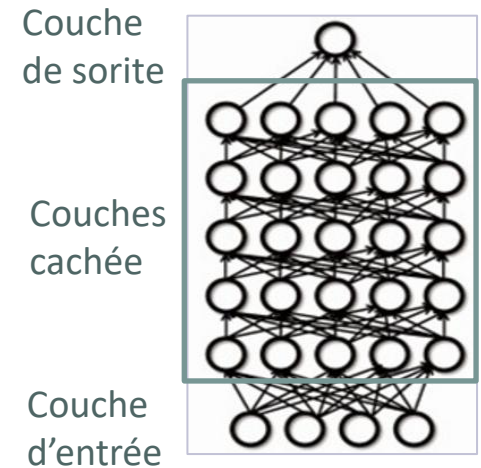
Perceptron simple vs. multicouches



Perceptron simple

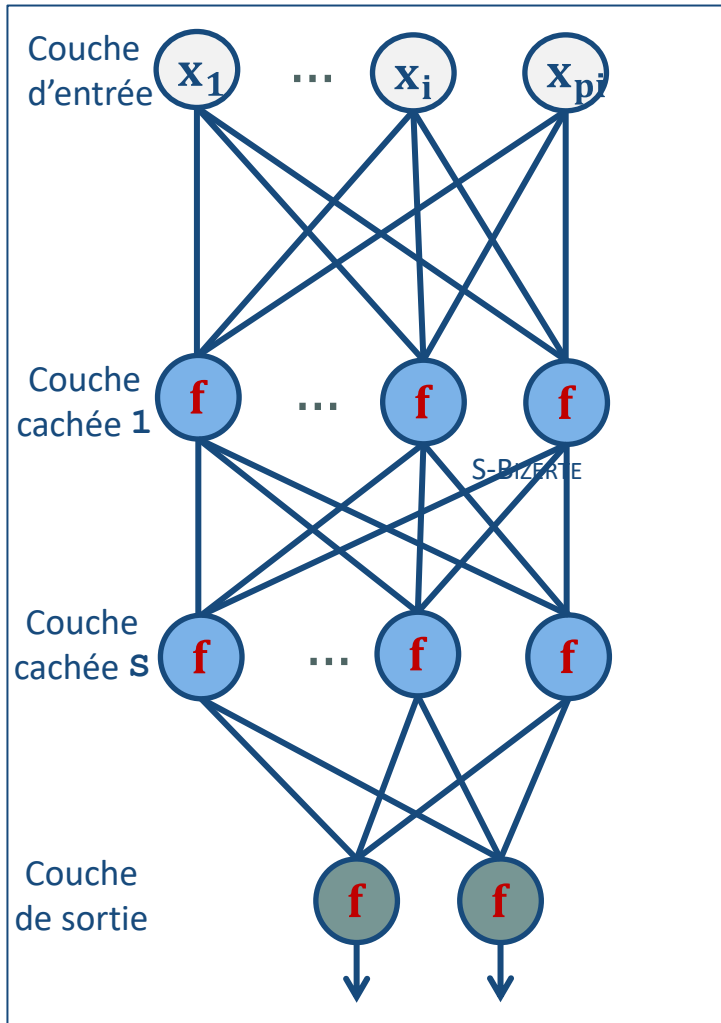


Perceptron multicouches (**MLP**)
= **Fully connected**



💡 Le terme "**profond**" dans "apprentissage profond" fait référence au nombre de couches cachée du réseau de neurones

Perceptron multicouches



x_1	x_2	...	x_p	Classe_d
				C1
				C2
				C3

Couche d'entrée = p Neurones

Une ou plusieurs couche(s) cachée(s)

Couche cachée 1 = k Neurones

Couche cachée S = m Neurones

Fully Connected & Séquentiel

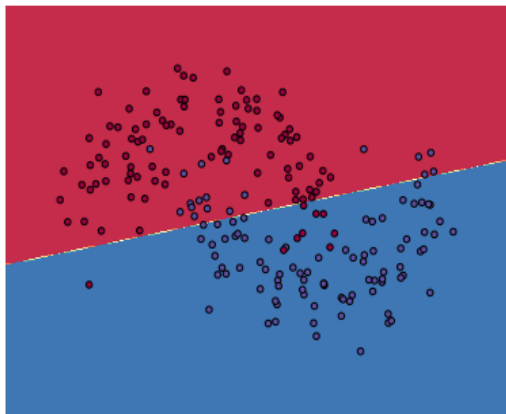
Couche de sortie = n Neurones = **Nombre de classe**

Fonctions d'activation **f non linéaire**

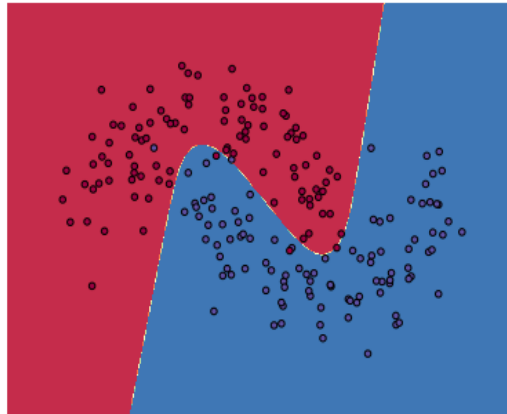
Impacts des couches cachées!

Pouvoir séparateur : permet de classer des formes **non linéairement séparables** ✓

↗ nombre de couches & ↗ nombre de neurones accroît le pouvoir de séparation



0 Couche cachée



3 couches cachées
chacun de 10 neurones



20 couches cachées
chacun de 100 neurones

Fonction d'activation non linéaire **Sigmoïde**

Étant donné p neurones ($x_1, \dots, x_i, x_p$) dans la couche d'entrée et k neurones ($Y_1, \dots, Y_j, Y_k$) dans la **couche cachée 1**

$$Y_j = \sum_{i=1}^p w_{ij} x_i + b_j$$

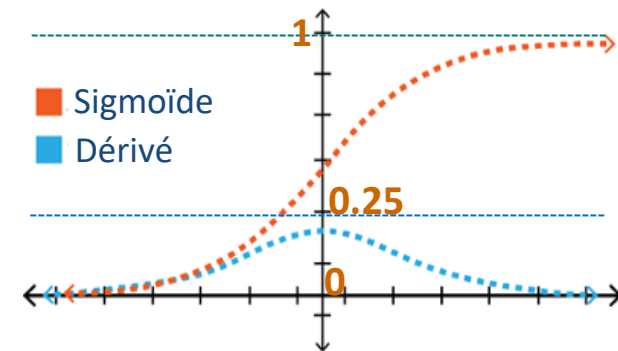
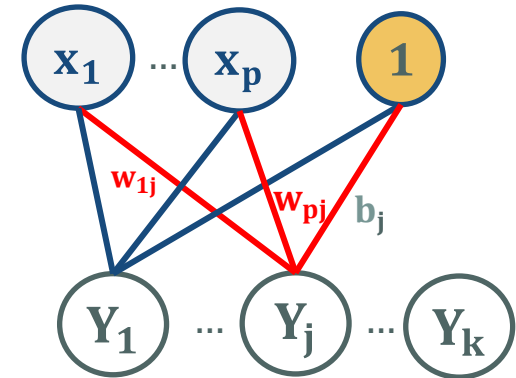
$$\text{Sigmoïde } f(Y_j) = \frac{1}{1 + e^{-Y_j}} \quad]0,1[$$

Dérivé de la fonction Sigmoïde

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = f(Y_j)(1 - f(Y_j)) \quad]0,0.25[$$



Classification binaire



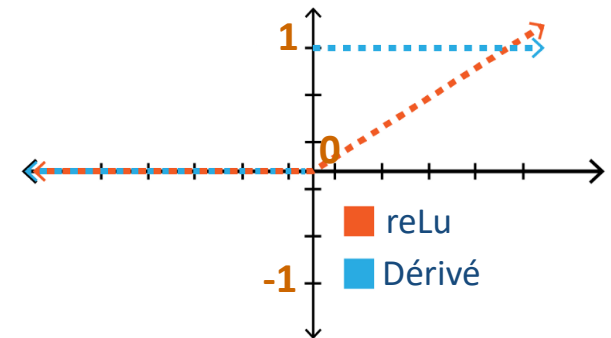
Autres fonctions d'activations Relu & ses variantes 1/2

Rectified Linear Unit (ReLU)

$$f(Y_j) = \max(0, Y_j) = \begin{cases} 0 & \text{si } Y_j < 0 \\ Y_j & \text{si } Y_j \geq 0 \end{cases} \quad [0, +\infty[$$

Dérivé de la fonction ReLU

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \begin{cases} 0 & \text{si } Y_j < 0 \\ 1 & \text{si } Y_j \geq 0 \end{cases} \quad 0 \text{ ou } 1$$

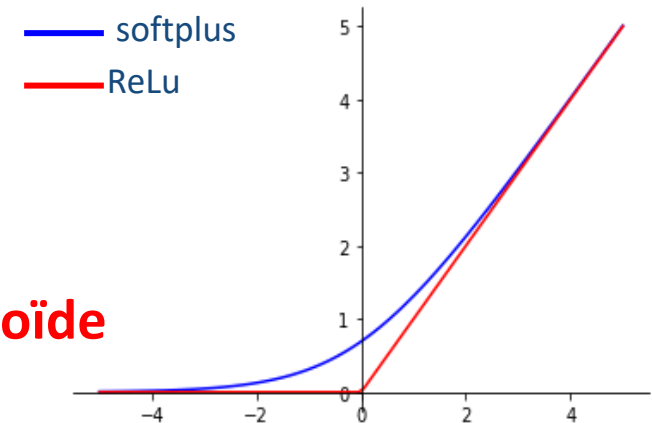


softplus

$$f(Y_j) = \ln(e^{Y_j} + 1) \quad] -\infty, +\infty[$$

Dérivé de la fonction softplus = fonction Sigmoide

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \frac{1}{1 + e^{-Y_j}} \quad]0, 1[$$

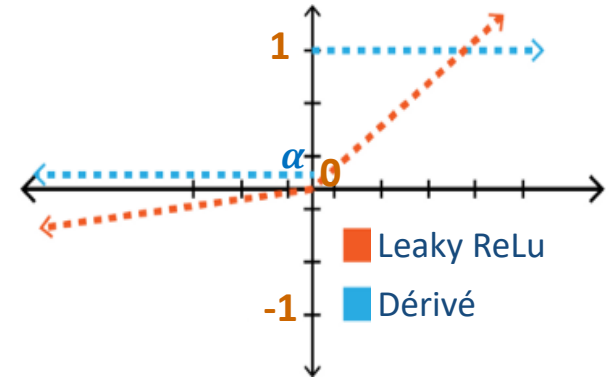


Autres fonctions d'activations Relu & ses variantes 2/2

$$\text{Leaky ReLu } f(Y_j) = \begin{cases} \alpha Y_j & \text{si } Y_j < 0 \\ Y_j & \text{si } Y_j \geq 0 \end{cases} \quad] -\infty, +\infty[$$

Dérivé de la fonction Leaky ReLu

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \begin{cases} \alpha & \text{si } Y_j < 0 \\ 1 & \text{si } Y_j \geq 0 \end{cases} \quad \alpha \text{ ou } 1$$

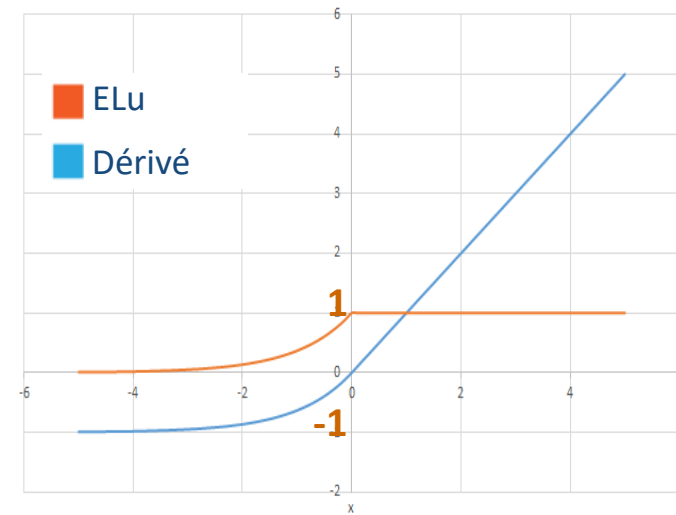


Exponential Linear Unit (ELu)

$$f(Y_j) = \begin{cases} \alpha (e^{Y_j} - 1) & \text{si } Y_j < 0 \\ Y_j & \text{si } Y_j \geq 0 \end{cases} \quad] -\infty, +\infty[$$

Dérivé de la fonction ELu

$$\frac{\partial f(Y_j)}{\partial Y_j} = f'(Y_j) = \begin{cases} f(Y_j) + \alpha & \text{si } Y_j < 0 \\ 1 & \text{si } Y_j \geq 0 \end{cases} \quad] -\infty, 1]$$



Autres fonctions d'activations **Softmax**

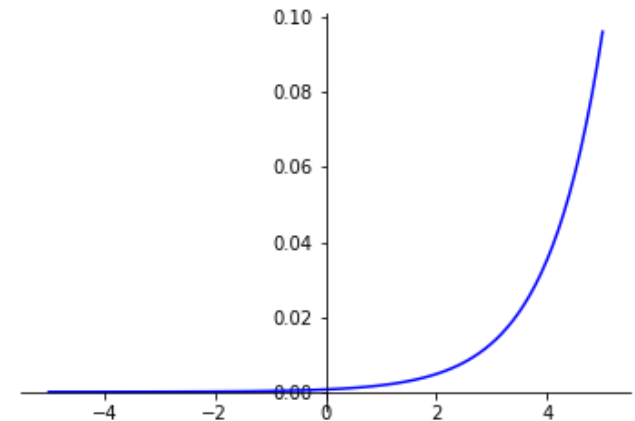
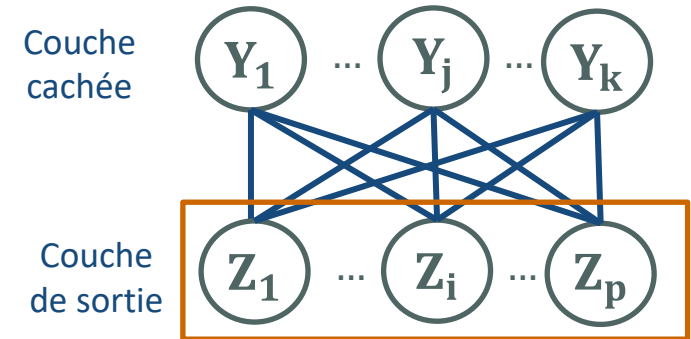
$$\text{softmax } f(\mathbf{Z}_i) = \frac{e^{Z_i}}{\sum_{a=1}^p e^{Z_a}}$$

Dérivé de la fonction softmax

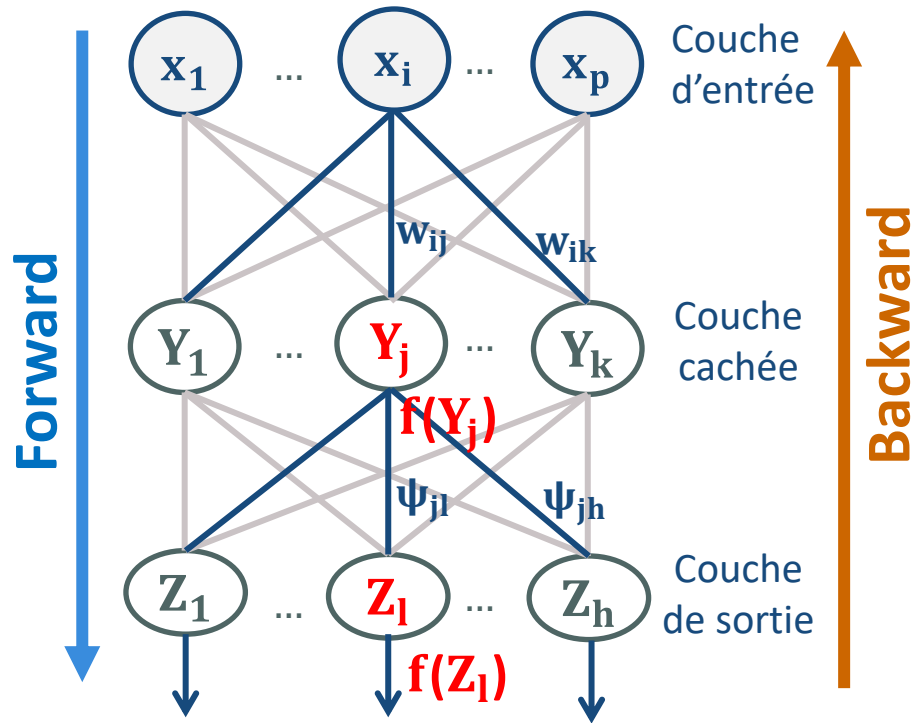
$$f'(\mathbf{Z}_i) = \begin{cases} f(\mathbf{Z}_i)(1 - f(\mathbf{Z}_i)) & \text{si } i = j \\ -f(\mathbf{Z}_i)f(\mathbf{Z}_j) & \text{si } i \neq j \end{cases}$$

✓ Utilisée pour activer les neurones de la couche de sortie

⚠ **Classification multi-classes**



Rétropropagation du gradient **Backward + Forward**



loss **E** ?

Forward : Calcul d'erreur **E**

$$f(Y_j) = f\left(\sum_{i=1}^p w_{ij} x_i + b_j\right)$$

$$f(Z_l) = f\left(\sum_{j=1}^k \psi_{jl} f(Y_j) + b_l\right) = \text{sortie}$$

E = ?

Backward = Descente de gradient

Minimiser **E** & Ajustement des poids

$$w = w - \eta \frac{\partial E}{\partial w}$$

Fonction du perte (**loss E**)

☰ Classification

☰ **Cross Entropy** h classes ($h > 2$) = h neurones dans la couche de sortie

$$E = \sum_{l=1}^h -d_l \ln(f(Z_l)) \quad \text{avec} \quad \begin{cases} h = \text{Nombre de classe} \\ d = \text{Classe désirée} \end{cases}$$

☰ **Binary Cross Entropy** 2 classes = 1 neurones dans la couche de sortie **Z**

$$E = -[d \ln(f(Z)) + (1 - d) \ln(1 - f(Z))]$$

☰ Régression : **Mean Square Error** = 1 neurones dans la couche de sortie **Z**

$$E = \frac{1}{M} \sum_{i=1}^M (d^{(i)} - f(Z)^{(i)})^2 \quad \text{avec} \quad \begin{cases} M = \text{Nombre d'observation} \\ d^{(i)} = \text{Classe désirée de l'observation } i \\ f(Z)^{(i)} = \text{classe prédite de l'observation } i \end{cases}$$

Descente de gradient **Backward**

Répéter

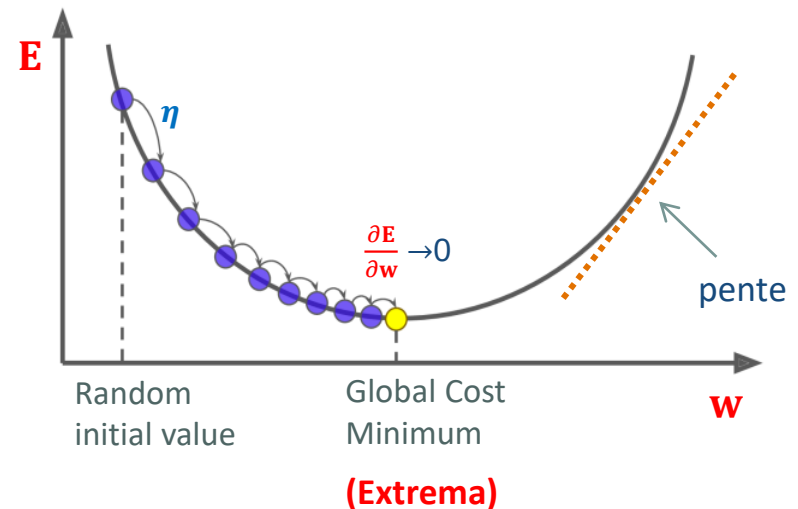
a. calculer le gradient (pente)

$$\nabla g = \frac{\partial E}{\partial \mathbf{w}}$$

b. Mettre à jour les poids

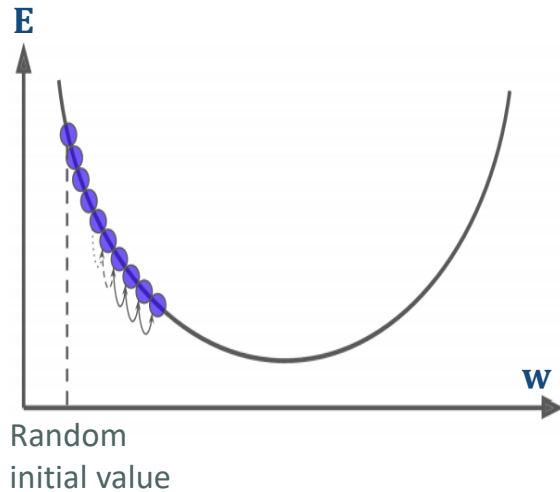
$$\mathbf{w} = \mathbf{w} - \eta \frac{\partial E}{\partial \mathbf{w}}$$

Jusqu'à Trouver le minimum
d'erreur (convergence)



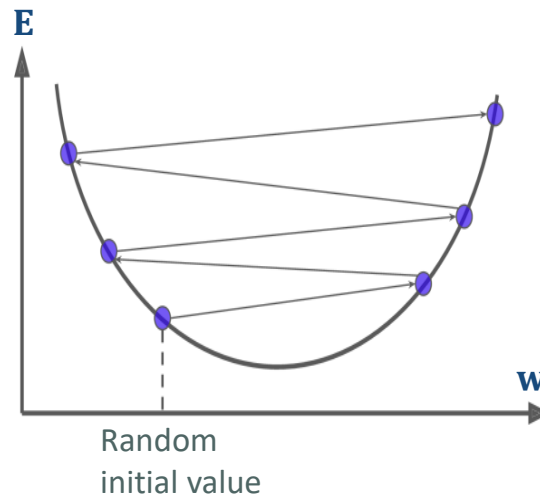
$$\frac{\partial E}{\partial w} < 0 \Rightarrow \text{pente descend vers la droite}$$
$$\frac{\partial E}{\partial w} > 0 \Rightarrow \text{pente monte vers la droite}$$

Problèmes de descente de gradient



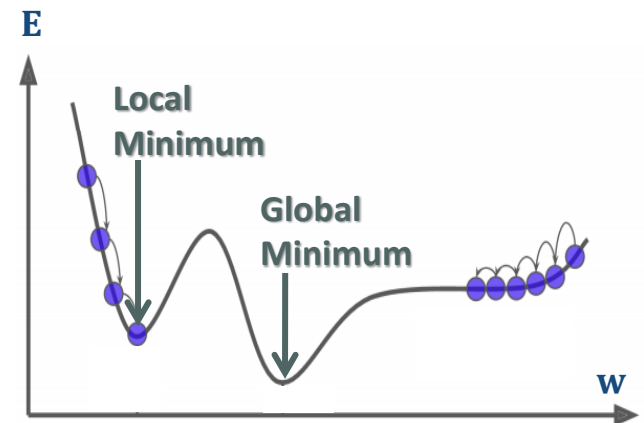
η EST TROP PETIT

- ⊖ Nombreuses itérations pour converger
- ⊖ Temps important de calcul



η EST TROP ÉLEVÉ

- ⊖ Sauter d'un côté à un autre
- ⊖ Diverger l'algorithme



TOMBER DANS LE MINIMUM LOCAL

- ⊖ Local Minimum > Global Minimum



Choix Learning rate $\eta = 0.001$

Activité (cas de classification binaire) 1/3

I- Forward (propagation avant)

1- Calcul de l'activation des neurones de la couche cachées

$$f(Y_1) = f\left(\sum_{i=1}^3 w_{i1} x_i + b_1\right) = \frac{1}{1 + e^{-Y_1}}$$

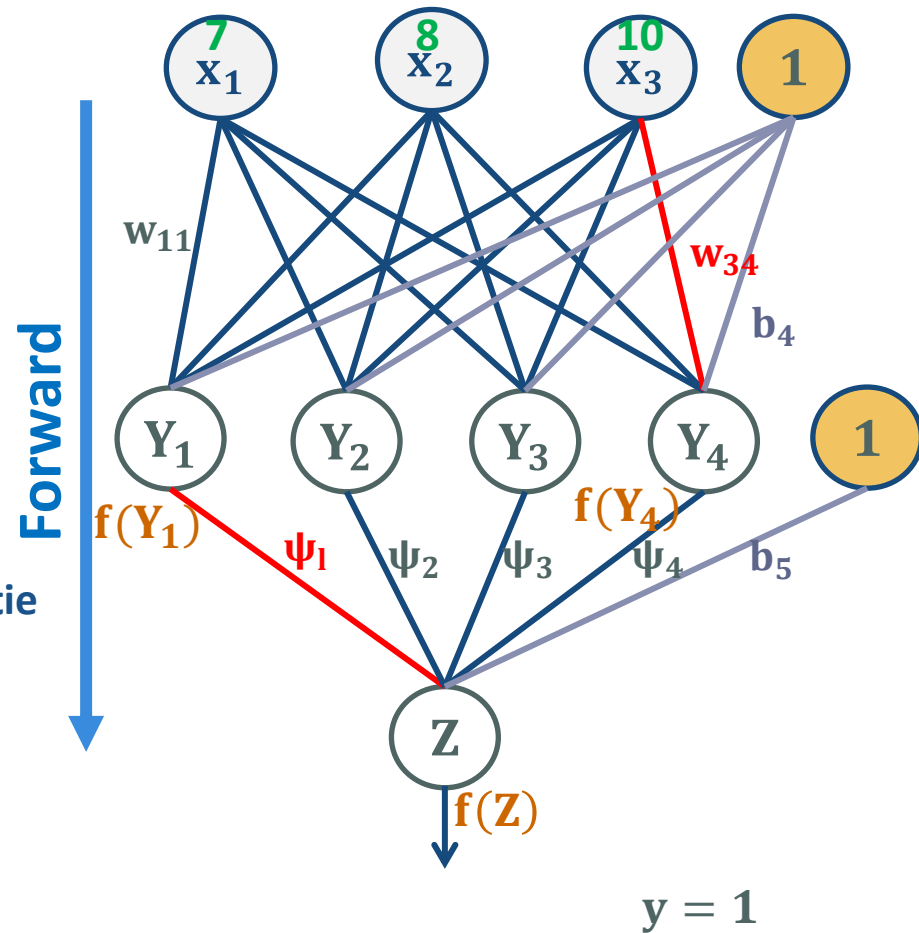
$$\vdots$$
$$f(Y_4) = f\left(\sum_{i=1}^3 w_{i4} x_i + b_4\right) = \frac{1}{1 + e^{-Y_4}}$$

2- Calcul de l'activation du neurone de la C. Sortie

$$f(Z) = f\left(\sum_{i=1}^4 \psi_i f(Y_i) + b_5\right) = \frac{1}{1 + e^{-Z}} = \text{sortie}$$

3- Calcul de loss **E** (Binary cross Entropy)

$$E = -[y \ln(f(Z)) + (1 - y) \ln(1 - f(Z))]$$



Activité (cas de classification binaire) 2/3

II- Backward : Descente de gradient

$$\psi_1? \quad \psi_1 = \psi_1 - \eta \frac{\partial E}{\partial \psi_1}$$

Chain rule

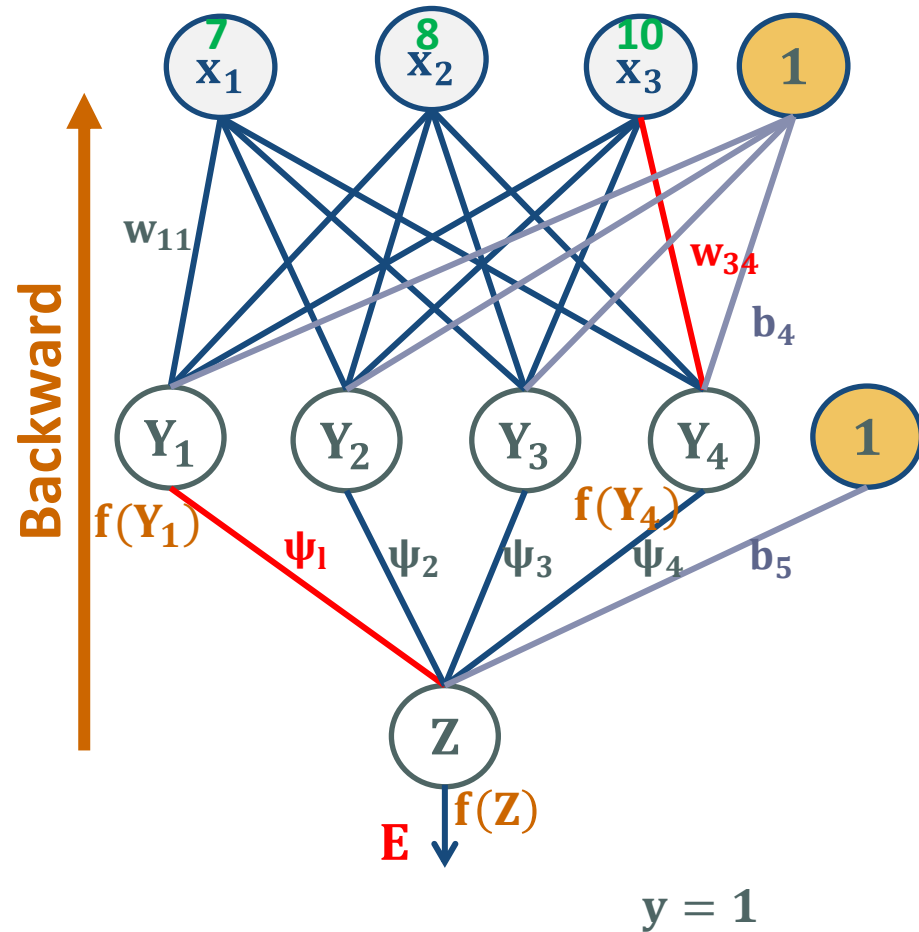
$$\frac{\partial E}{\partial \psi_1} = \frac{\partial E}{\partial f(Z)} \frac{\partial f(Z)}{\partial Z} \frac{\partial Z}{\partial \psi_1}$$



$$\frac{\partial Z}{\partial \psi_1} = f(Y_1)$$

$$\frac{\partial f(Z)}{\partial Z} = f'(Z) = f(Z)(1 - f(Z))$$

$$\frac{\partial E}{\partial f(Z)} = ?$$



Activité (cas de classification binaire) 3/3

$$\frac{\partial \ln(x)}{\partial x} = \frac{1}{x}$$

$$\frac{\partial \ln(1-x)}{\partial x} = -\frac{1}{1-x}$$

$$\frac{\partial E}{\partial f(Z)} = \frac{\partial [-y \ln(f(Z)) - (1-y) \ln(1-f(Z))]}{\partial f(Z)} =$$

$\downarrow$ $\downarrow$

$-y \frac{1}{f(Z)} \quad + (1-y) \frac{1}{1-f(Z)}$

$$\frac{\partial E}{\partial f(Z)} = -\frac{y}{f(Z)} + \frac{1-y}{1-f(Z)}$$

Chain rule

$$\frac{\partial E}{\partial \psi_1} = \frac{\partial E}{\partial f(Z)} \frac{\partial f(Z)}{\partial Z} \frac{\partial Z}{\partial \psi_1}$$

$$\frac{\partial Z}{\partial \psi_1} = f(Y_1)$$

$$\frac{\partial f(Z)}{\partial Z} = f'(Z) = f(Z)(1-f(Z))$$

$$\frac{\partial E}{\partial f(Z)} = ?$$

$$E = -[y \ln(f(Z)) + (1-y) \ln(1-f(Z))]$$

Epoch, size_batch & Iteration



ÉPOQUE (epoch)

Passage sur l'ensemble de toutes les **n** observations de la base d'apprentissage



Size_batch

Nombre total d'observations d'apprentissage présents dans un seul batch



Iteration

Nombre des batches nécessaires pour compléter une époque

Variantes de descente de gradient (1/2)

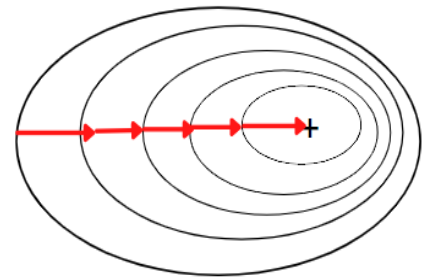
✓ Descente de gradient classique Batch Gradient Descent (BGD)

— Mettre à jour les poids après avoir fait passer la totalité des observations par époque

$$\mathbf{w}_i = \mathbf{w}_i - \eta \sum_{j=1}^N \frac{\partial E}{\partial \mathbf{w}_i}$$

N = Nombre d'observation

- 😊 Stable
- 😞 Mise à jour lent des poids
- 😞 Facile à tomber dans le minimum local

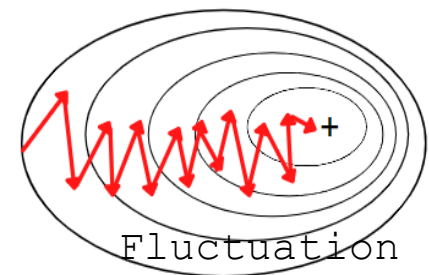


✓ Descente de gradient stochastique Stochastic Gradient Descent (SGD)

— Mettre à jour les poids au passage de chaque observation (indépendamment du nombre d'époque)

$$\mathbf{w}_i = \mathbf{w}_i - \eta \frac{\partial E}{\partial \mathbf{w}_i}$$

- 😊 Mise à jour rapide des poids
- 😞 Instable
- 😞 Difficile de converger vers l'extrema



Variantes de descente de gradient (2/2)

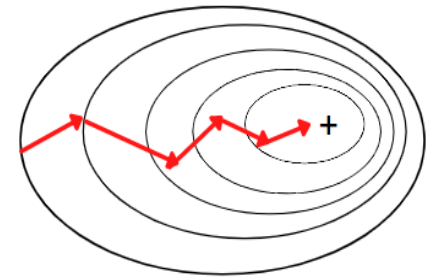
✓ Descente de gradient Mini-batch Mini-Batch Gradient Descent (MBGD)

— Mettre à jour par blocs d'observations (`batch-size`)

$$\mathbf{w}_i = \mathbf{w}_i - \eta \sum_{j=1}^K \sum_{j=1}^M \frac{\partial E}{\partial \mathbf{w}_i}$$

K = Nombre de batch=Itération

M = Nombre d'observation dans un
batch=`batch-size`



- 😊 Convergence plus rapide que BGD
- 😞 Sensibilité à la taille du lot (`batch_size`)
- 😞 Facile à tomber dans le minimum local



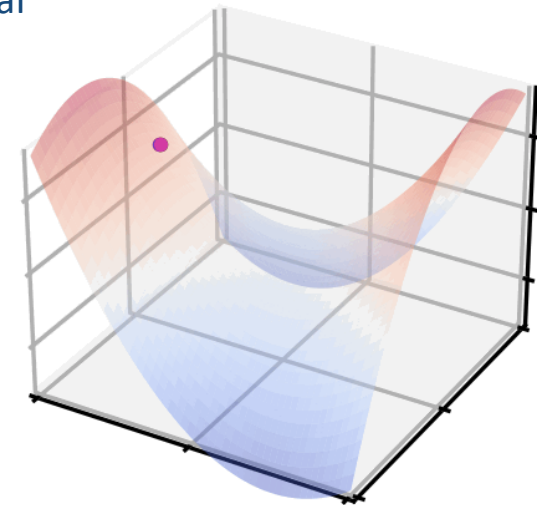
Autres variantes de descente de gradient **Optimiseur**

✓ Optimiser la manière d'ajustement de poids

- Accélérer la convergence tout en minimisant la fonction de perte
- Éviter ou dépasser les minimums locaux : atteindre le minimum global

📄 Optimiseurs les plus courants

- Batch Gradient Descent (BGD)
- Stochastic Gradient Descent (SGD)
- Mini-Batch Gradient Descent (MBGD)
- Adaptive Gradient Algorithm (Adagrad)
- Adaptive Learning Rate Method (Adadel Δ)
- Root Mean Square Propagation (RMSprop)
- Adaptive Moment Estimation (Adam)



MERCI DE VOTRE ATTENTION

شكرا على المتابعة